Memoria Práctica 3

Just Eat



Tecnologías Web

4 / 05 / 2026

Profesor:

José Carlos Sancho Núñez

Autor:

Francisco Montero Sánchez

Índice

1 - Introducción 3
2 - Login previo 3
3 - Requisitos mínimos 4
3.1 - Requisito 1. Registro y acceso de usuarios 4
Prueba 1.1 - Consultar usuario autenticado 4
Prueba 1.2 - Consultar un usuario ajeno 5
Prueba 1.3 - Editar usuario autenticado 6
Prueba 1.4 - Crear usuario de prueba 7
Prueba 1.5 - Borrar usuario de prueba 8
3.2 - Requisito 2. Gestión de restaurantes 9
Prueba 2.1 - Listar todos los restaurantes 9
Prueba 2.2 - Listar restaurantes propios 10
Prueba 2.3 - Consultar detalle de restaurante 11
Prueba 2.4 - Crear restaurante 12
Prueba 2.5 - Editar restaurante creado 13
Prueba 2.6 - Intentar borrar restaurante ajeno 15
Prueba 2.7 - Borrar restaurante creado 16
3.3 - Requisito 3. Gestión de platos de un restaurante 17
Prueba 3.1 - Listar platos de un restaurante 17
Prueba 3.2 - Crear plato 18
Prueba 3.3 - Intentar crear plato en restaurante ajeno 19
Prueba 3.4 - Editar plato creado 21
Prueba 3.5 - Borrar plato creado 22
3.4 - Requisito 4. Gestión de pedidos 23
Prueba 4.1 - Crear pedido 23
Prueba 4.2 - Consultar histórico de pedidos 25
Prueba 4.3 - Consultar pedido creado 26
3.5 - Requisito 5. Valoración 27
Prueba 5.1 - Consultar valoraciones de restaurante 27
Prueba 5.2 - Crear o actualizar valoración 28
Prueba 5.3 - Consultar restaurantes ordenados por valoración 29
3.6 - Requisito 6. Buscar restaurante 30
Prueba 6.1 - Buscar por título del restaurante 30
Prueba 6.2 - Buscar restaurantes por localidad 31
3.7 - Requisito 7. Estado del restaurante 32
Prueba 7.1 - Cambiar estado de restaurante 32
Prueba 7.2 - Intentar cambiar estado de restaurante ajeno 33
Prueba 7.3 - Filtrar restaurantes que aceptan pedidos 34
4 - Requisitos extra 35
4.0 - Requisito Extra. Inicio de sesión REST 35
Prueba RE0.1 - Iniciar sesión REST 36

4.1 - Requisito Extra 1. Gestión de categorías 37
Prueba RE1.1 - Listar categorías 37
Prueba RE1.2 - Consultar categoría 38
Prueba RE1.3 - Crear categoría de prueba 39
Prueba RE1.4 - Editar categoría de prueba 40
Prueba RE1.5 - Agregar categoría a restaurante propio 41
Prueba RE1.6 - Listar categorías de restaurante 42
Prueba RE1.7 - Intentar asociar categoría a restaurante ajeno 43
Prueba RE1.8 - Consultar restaurantes por categoría 44
Prueba RE1.9 - Quitar categoría de restaurante 45
Prueba RE1.10 - Borrar categoría de prueba 46
4.2 - Requisito Extra 2. Restaurantes favoritos 47
Prueba RE2.1 - Añadir restaurante a favoritos 47
Prueba RE2.2 - Listar favoritos 48
Prueba RE2.3 - Intentar añadir favorito duplicado 49
Prueba RE2.4 - Consultar favoritos de usuario ajeno 50
Prueba RE2.5 - Quitar restaurante de favoritos 51
4.3 - Requisito Extra 3. Búsqueda avanzada 52
Prueba RE3.1 - Filtrar por categoría 52
Prueba RE3.2 - Filtrar por comentarios 53
Prueba RE3.3 - Filtrar por plato 54
Prueba RE3.4 - Filtrar por rango de precios 55
Prueba RE3.5 - Filtros combinados 56
4.4 - Requisito Extra 4. Restaurantes relacionados 57
Prueba RE4.1 - Consultar restaurantes relacionados 57
Prueba RE4.2 - Consultar relacionados de restaurante inexistente 58
4.9 - Requisito Extra 9. Panel de redes sociales 59
Prueba RE9.1 - Crear restaurante con perfil social 59
Prueba RE9.2 - Consultar redes sociales de restaurante 60
Prueba RE9.3 - Editar red social de restaurante 61
Prueba RE9.4 - Eliminar red social de restaurante 62

1 - Introducción

Este documento recoge las evidencias de prueba de los endpoints implementados en la API REST de JustEatDAO_Rest. Las pruebas se han organizado por requisitos funcionales mínimos de la práctica, indicando para cada una el usuario utilizado, el método HTTP, el endpoint probado, el objetivo de la prueba y el código de respuesta esperado.

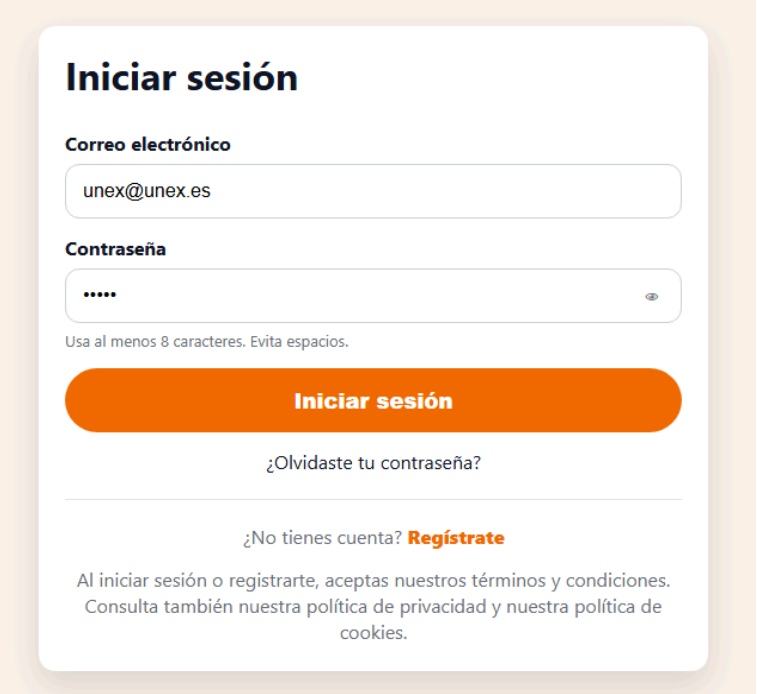
Las pruebas se han preparado en el fichero JSON exportado:

JustEatDAO_Rest_Pruebas.json

2 - Login previo

Antes de ejecutar las pruebas de los endpoints REST se realiza el inicio de sesión mediante el servlet de login del proyecto, igual que en la práctica 2. De esta forma se crea la sesión del usuario en el servidor y las llamadas posteriores a la API pueden comprobar qué usuario está autenticado.

En esta prueba se inicia sesión con el usuario José Carlos Sancho, usando el email unex@unex.es y la contraseña 12345. Después de hacer login, se ejecutan las peticiones REST para poder probar los distintos endpoints.



JustEat Club



3 - Requisitos mínimos

3.1 - Requisito 1. Registro y acceso de usuarios

Llamadas a la API disponibles:

GET /api/usuarios/{id}
POST /api/usuarios
PUT /api/usuarios/{id}
DELETE /api/usuarios/{id}

Prueba 1.1 - Consultar usuario autenticado

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

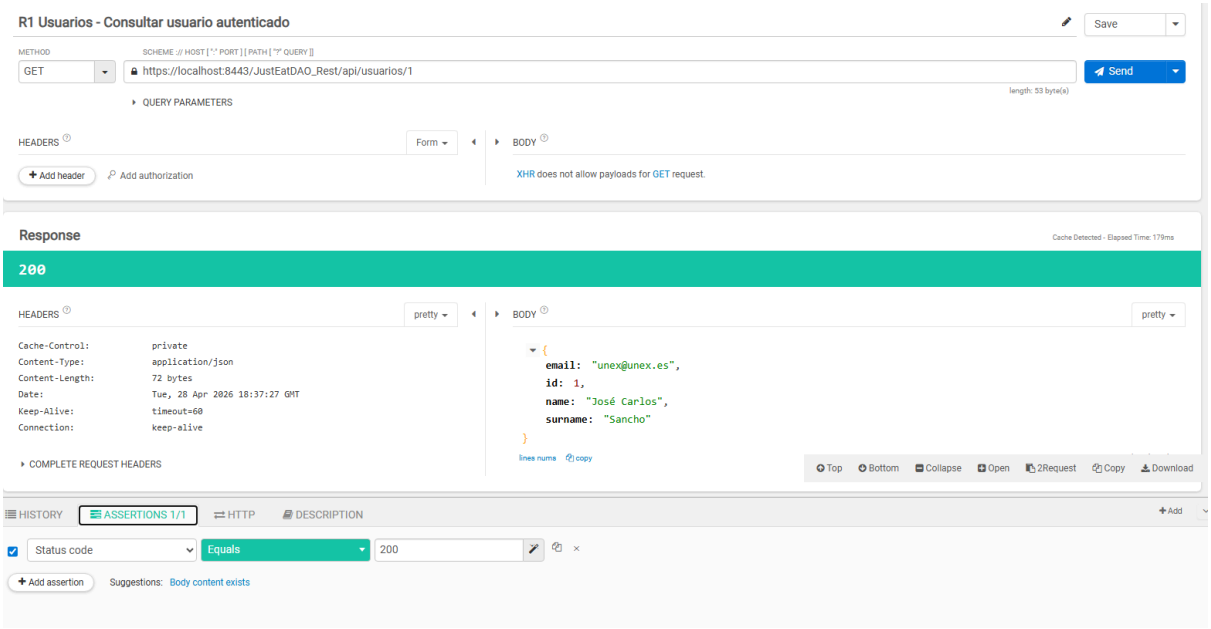
Método:
GET

Endpoint:
/api/usuarios/1

Objetivo:
Comprobar que el usuario autenticado puede consultar sus propios datos.

Resultado esperado:
200 OK

Evidencia:



Prueba 1.2 - Consultar un usuario ajeno

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

Usuario al que se intenta acceder:
Pilar Redondo
ID: 2

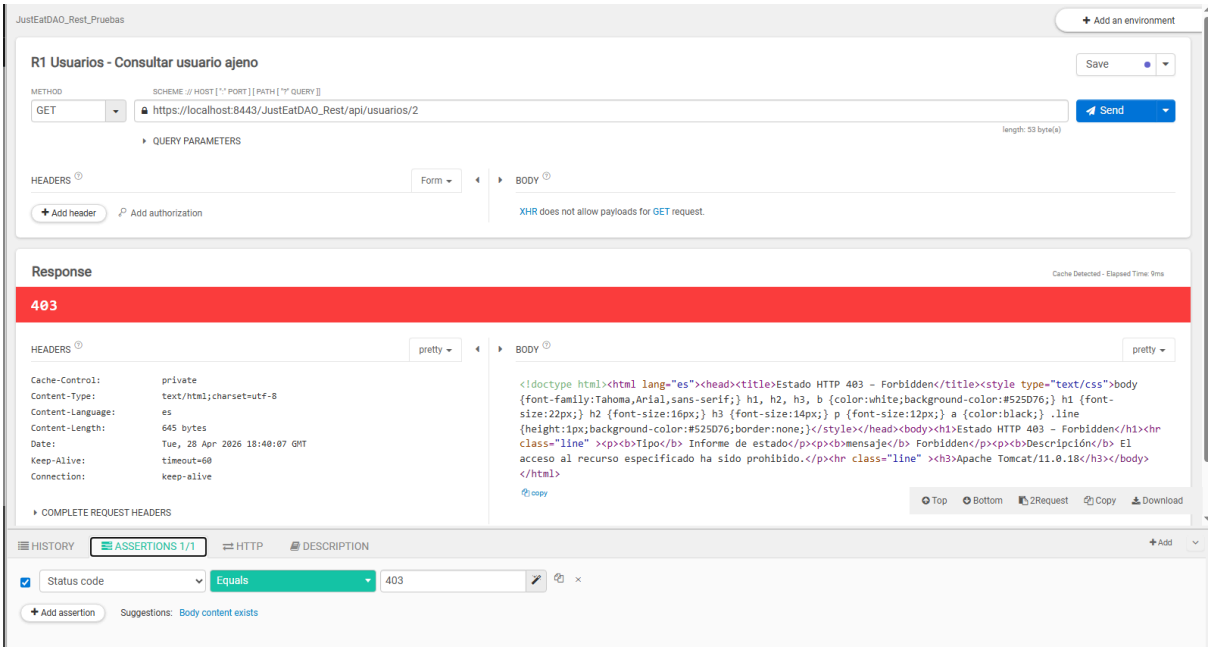
Método:
GET

Endpoint:
/api/usuarios/2

Objetivo:
Comprobar que un usuario autenticado no puede consultar los datos de otro usuario.

Resultado esperado:
403 Forbidden

Evidencia:



Prueba 1.3 - Editar usuario autenticado

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

Método:
PUT

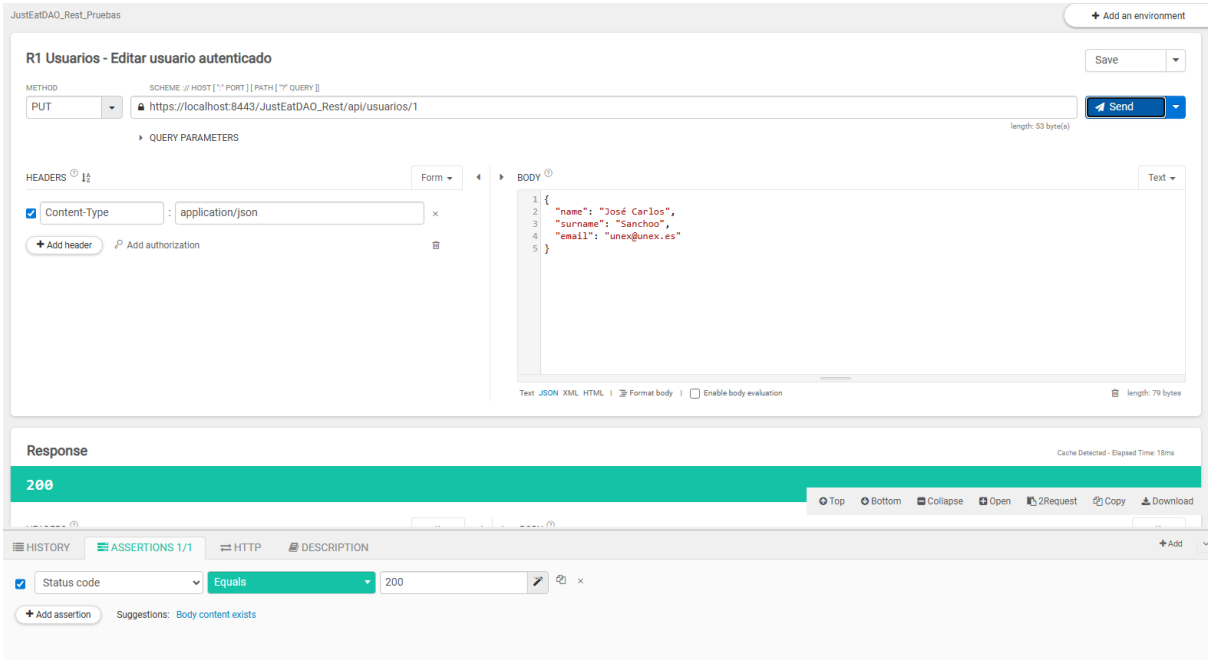
Endpoint:
/api/usuarios/1

Cuerpo:
```
{
  "name": "José Carlos",
  "surname": "Sanchoo",
  "email": "unex@unex.es"
}
```

Objetivo:
Comprobar que el usuario autenticado puede editar sus propios datos.

Resultado esperado:
200 OK

Evidencia:



Prueba 1.4 - Crear usuario de prueba

Usuario creado:
Fran Prueba
Email: prueba@prueba.com

Método:
POST

Endpoint:
/api/usuarios

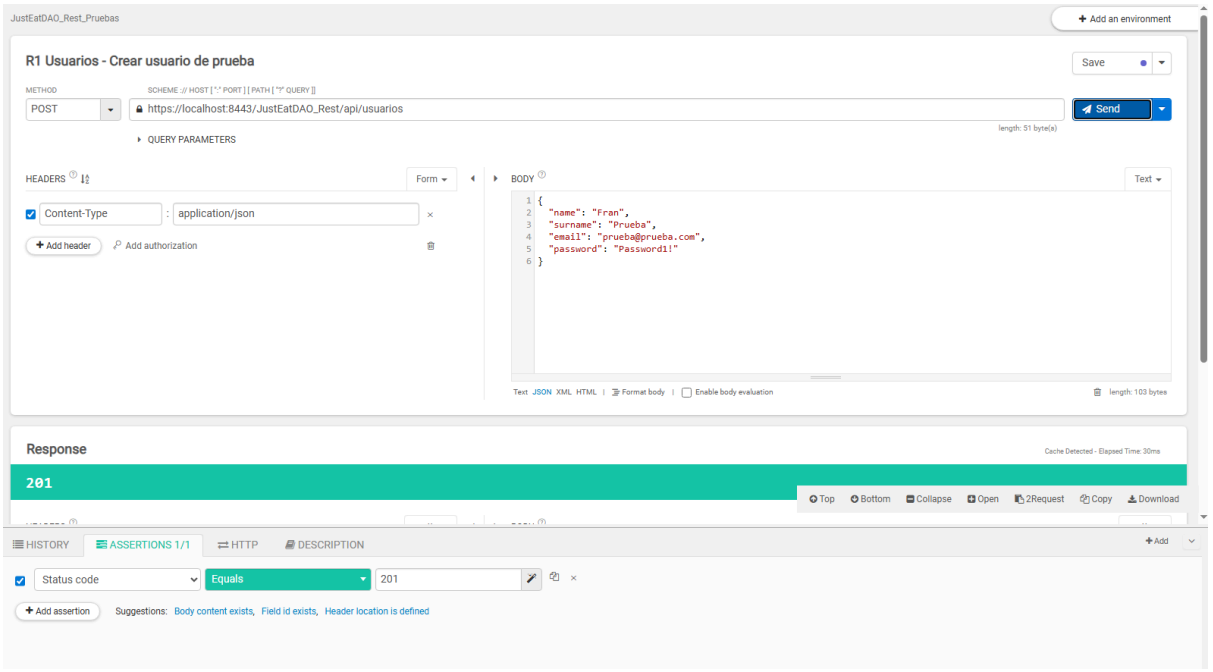
Cuerpo:
{
"name": "Fran",
"surname": "Prueba",
"email": "prueba@prueba.com",
"password": "Password1!"
}

Objetivo:
Comprobar que la API permite registrar un nuevo usuario.

Resultado esperado:

201 Created

Evidencia:



Prueba 1.5 - Borrar usuario de prueba

Usuario usado:
Fran Prueba
Email: prueba@prueba.com

Método:
DELETE

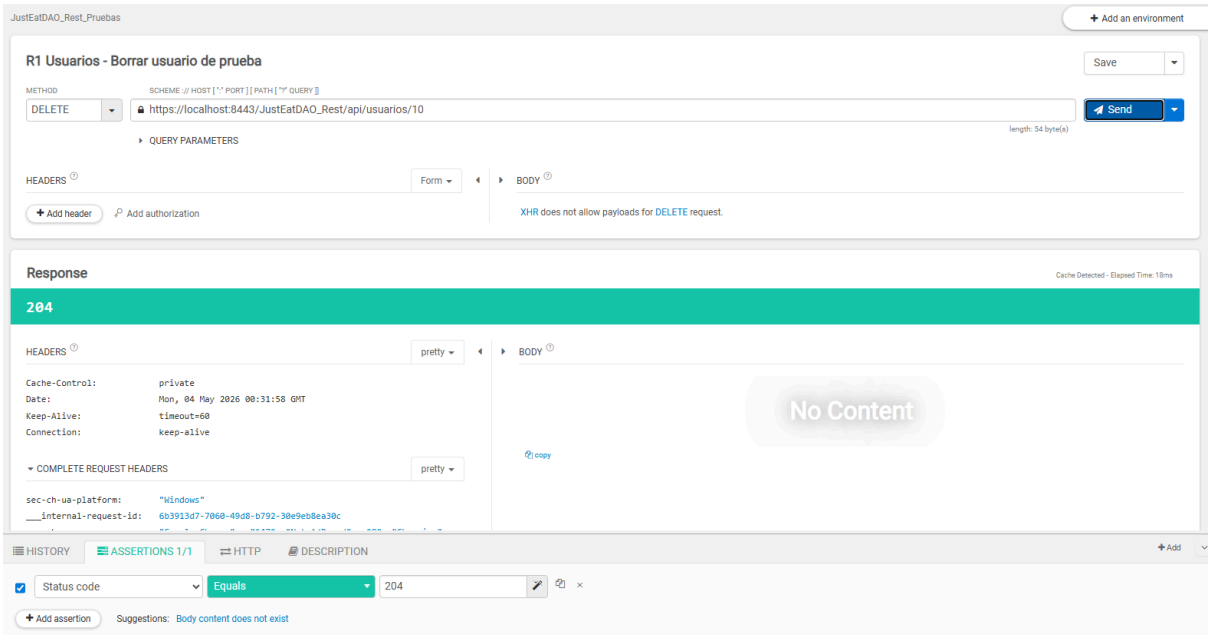
Endpoint:
/api/usuarios/10

Nota:
Si el ID devuelto al crear el usuario no es 10, se sustituye por el ID real devuelto en la respuesta del POST.

Objetivo:
Comprobar que la API permite eliminar el usuario autenticado. Se utiliza un usuario de prueba para no borrar usuarios reales de la BD base.

Resultado esperado:
204 No Content

Evidencia:



3.2 - Requisito 2. Gestión de restaurantes

Llamadas a la API disponibles:

GET /api/restaurantes/all
GET /api/restaurantes
GET /api/restaurantes/{id}
POST /api/restaurantes
PUT /api/restaurantes/{id}
DELETE /api/restaurantes/{id}

Nota: Debido a que en el modelo la entidad Restaurante no tiene una lista de categorías, se tuvo que crear una DTO para poder pasar los restaurantes y la lista de categorías por el mismo body.

Prueba 2.1 - Listar todos los restaurantes

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

Método:

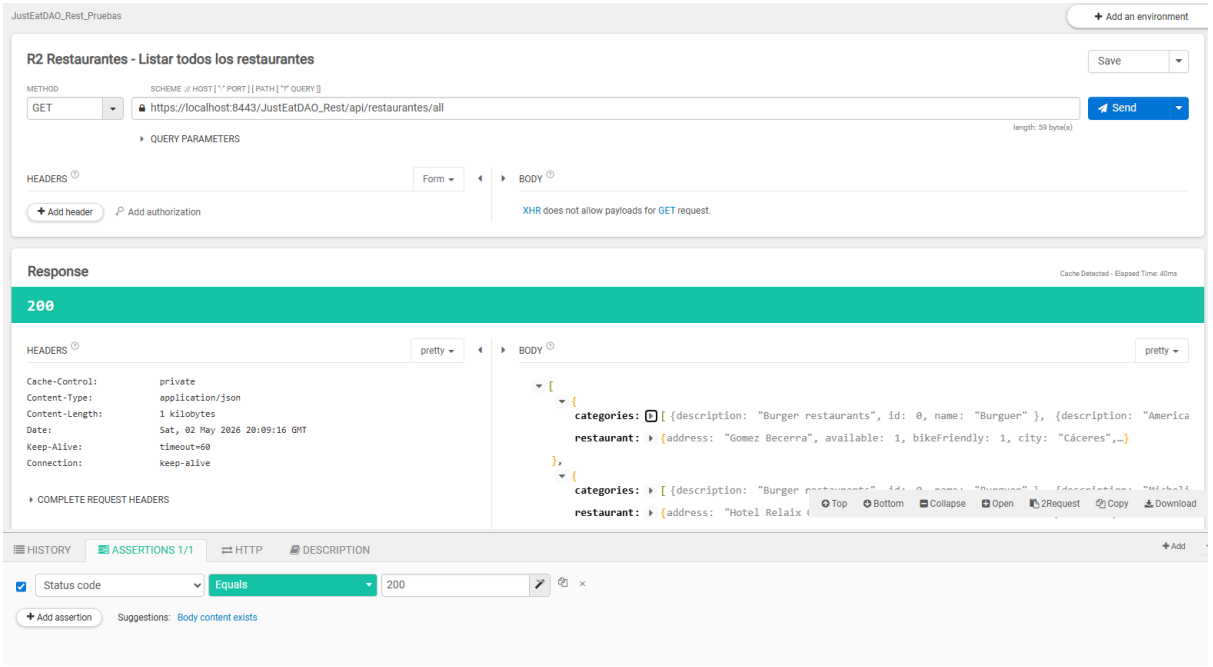
GET

Endpoint:
/api/restaurantes/all

Objetivo:
Comprobar que un usuario autenticado puede consultar el listado general de restaurantes.

Resultado esperado:
200 OK

Evidencia:



Prueba 2.2 - Listar restaurantes propios

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

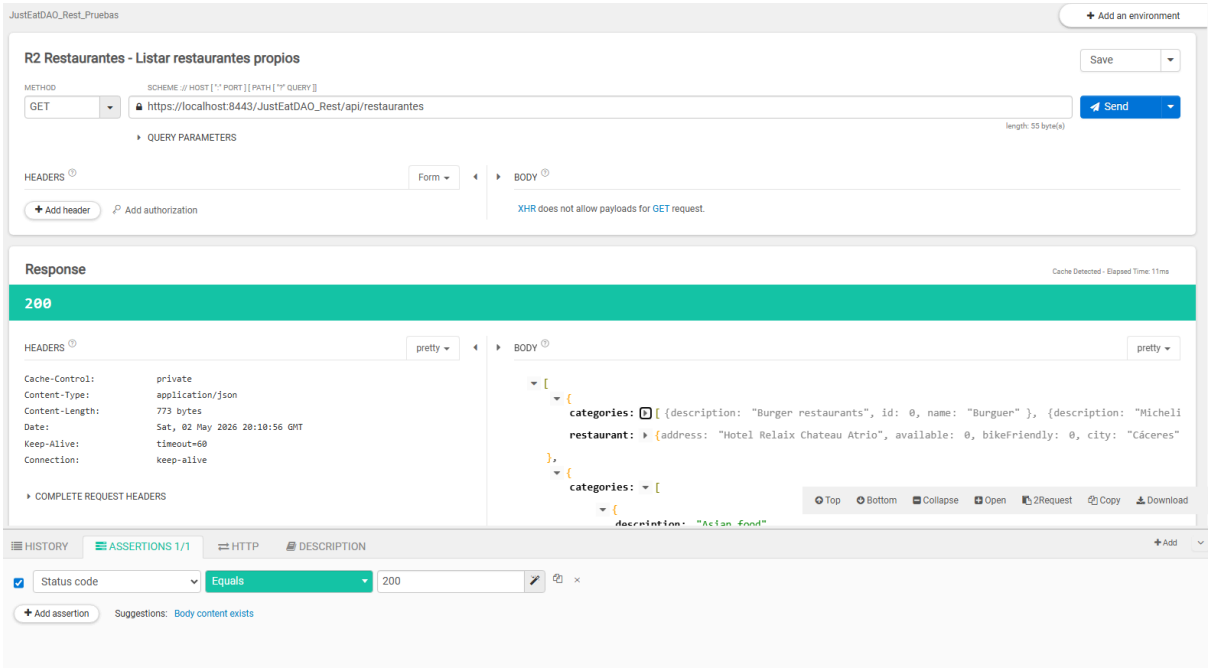
Método:
GET

Endpoint:
/api/restaurantes

Objetivo:
Comprobar que el usuario autenticado puede consultar los restaurantes que gestiona como propietario.

Resultado esperado:
200 OK

Evidencia:



Prueba 2.3 - Consultar detalle de restaurante

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

Restaurante consultado:
Atrio
ID: 1

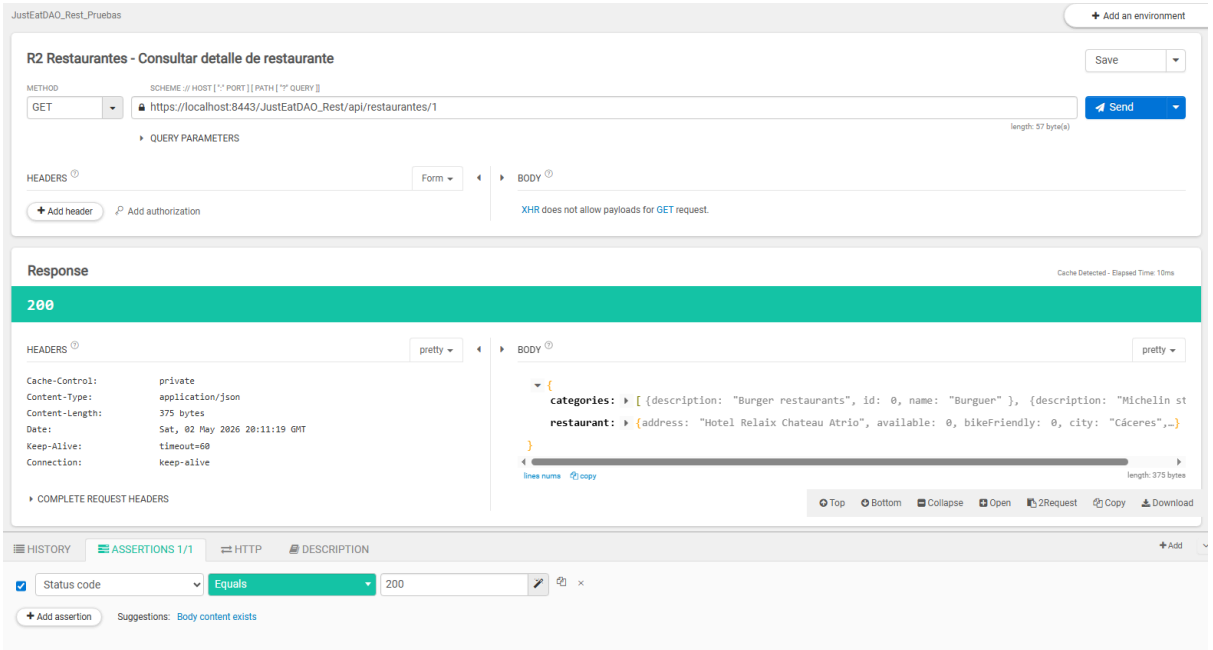
Método:
GET

Endpoint:
/api/restaurantes/1

Objetivo:
Comprobar que se puede consultar el detalle de un restaurante existente.

Resultado esperado:
200 OK

Evidencia:



Prueba 2.4 - Crear restaurante

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

Restaurante creado:
Restaurante de prueba

Método:
POST

Endpoint:
/api/restaurantes

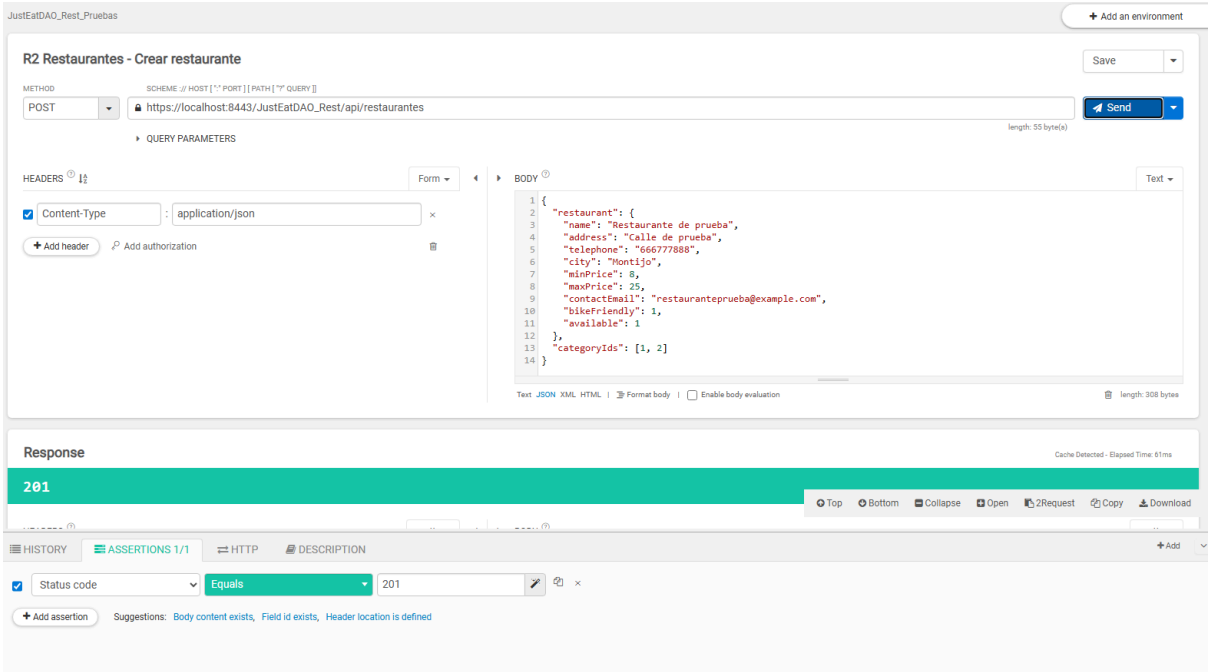
Cuerpo:

```
{
 "restaurant": {
  "name": "Restaurante de prueba",
  "address": "Calle de prueba",
  "telephone": "666777888",
  "city": "Montijo",
  "minPrice": 8,
  "maxPrice": 25,
  "contactEmail": "restauranteprueba@example.com",
  "bikeFriendly": 1,
  "available": 1
 },
 "categoryIds": [1, 2]
}
```


Objetivo:
Comprobar que el usuario autenticado puede crear un restaurante propio.

Resultado esperado:
201 Created

Evidencia:



Prueba 2.5 - Editar restaurante creado

Usuario usado:

José Carlos Sancho
Email: unex@unex.es
ID: 1

Restaurante editado:
Restaurante Prueba Fran

Método:
PUT

Endpoint:
/api/restaurantes/16

Nota:
Si el ID devuelto al crear el restaurante no es 16, se sustituye por el ID real devuelto en la respuesta del POST.

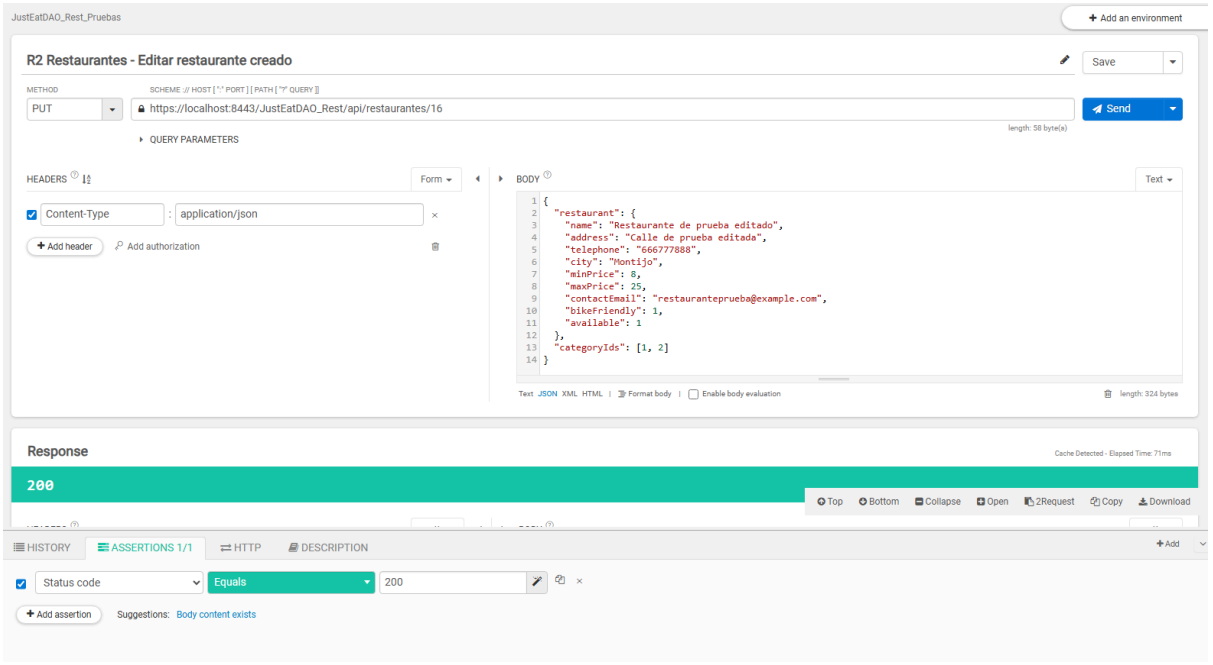
Cuerpo:

```
{
  "restaurant": {
    "name": "Restaurante de prueba editado",
    "address": "Calle de prueba editada",
    "telephone": "666777888",
    "city": "Montijo",
    "minPrice": 8,
    "maxPrice": 25,
    "contactEmail": "restauranteprueba@example.com",
    "bikeFriendly": 1,
    "available": 1
  },
  "categoryIds": [1, 2]
}
```


Objetivo:
Comprobar que el propietario puede modificar los datos de su restaurante.

Resultado esperado:
200 OK

Evidencia:



Prueba 2.6 - Intentar borrar restaurante ajeno

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

Restaurante ajeno:
Burger King
ID: 0
Propietario: admin@admin.es

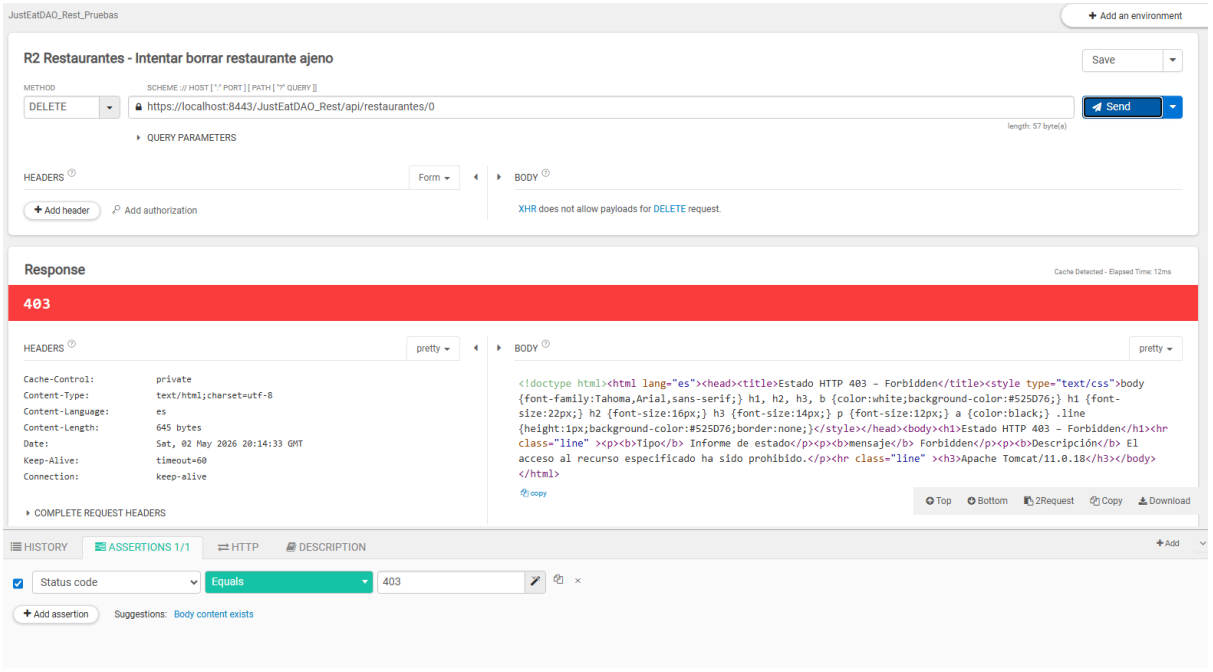
Método:
DELETE

Endpoint:
/api/restaurantes/0

Objetivo:
Comprobar que un usuario no puede eliminar un restaurante que pertenece a otro usuario.

Resultado esperado:
403 Forbidden

Evidencia:



Prueba 2.7 - Borrar restaurante creado

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

Restaurante borrado:
Restaurante de prueba editado

Método:
DELETE

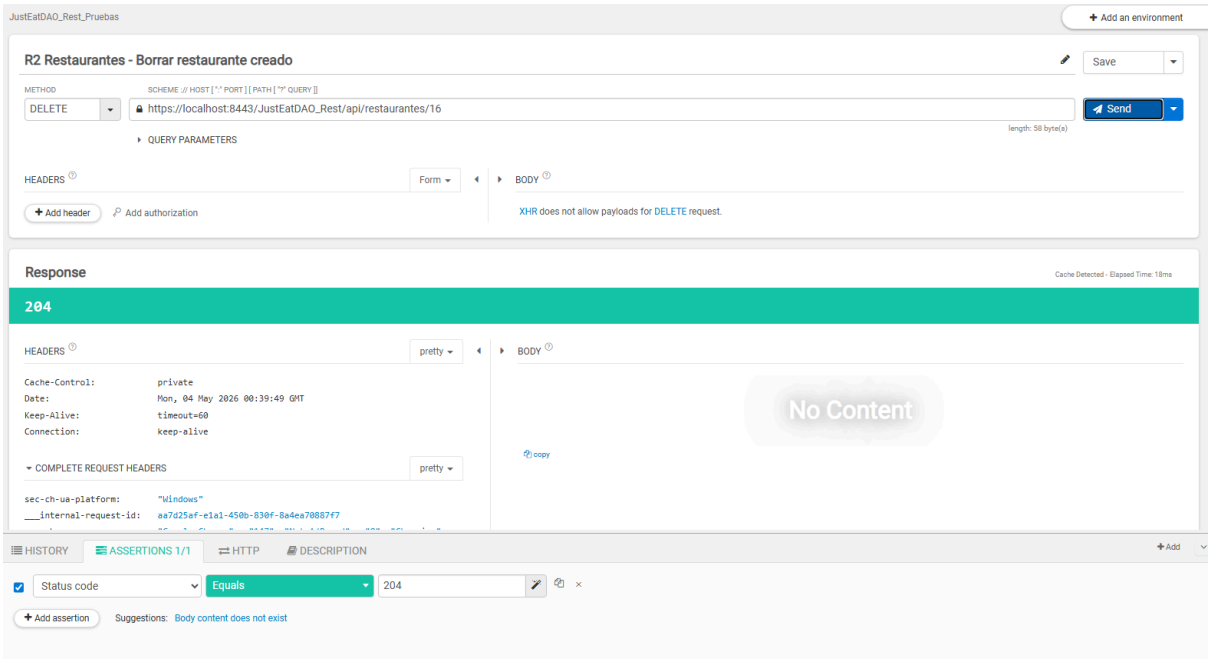
Endpoint:
/api/restaurantes/16

Nota:
Si el ID del restaurante creado no es 16, se sustituye por el ID real devuelto en la respuesta del POST.

Objetivo:
Comprobar que el propietario puede eliminar su propio restaurante.

Resultado esperado:
204 No Content

Evidencia:



3.3 - Requisito 3. Gestión de platos de un restaurante

Llamadas a la API disponibles:

GET /api/restaurantes/{id}/platos
POST /api/restaurantes/{id}/platos
PUT /api/platos/{id}
DELETE /api/platos/{id}

Prueba 3.1 - Listar platos de un restaurante

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

Restaurante:
Atrio
ID: 1

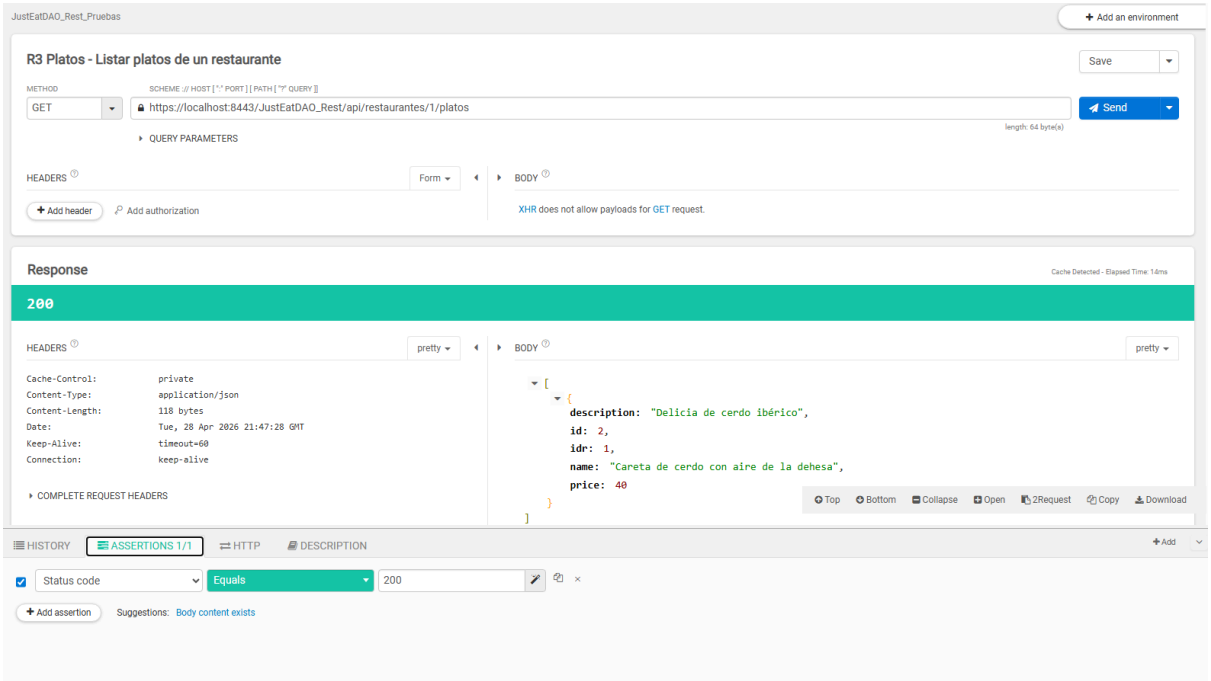
Método:
GET

Endpoint:
/api/restaurantes/1/platos

Objetivo:
Comprobar que se pueden consultar los platos asociados a un restaurante.

Resultado esperado:
200 OK

Evidencia:



Prueba 3.2 - Crear plato

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

Restaurante:
Atrio
ID: 1

Plato creado:

Plato de prueba

Método:
POST

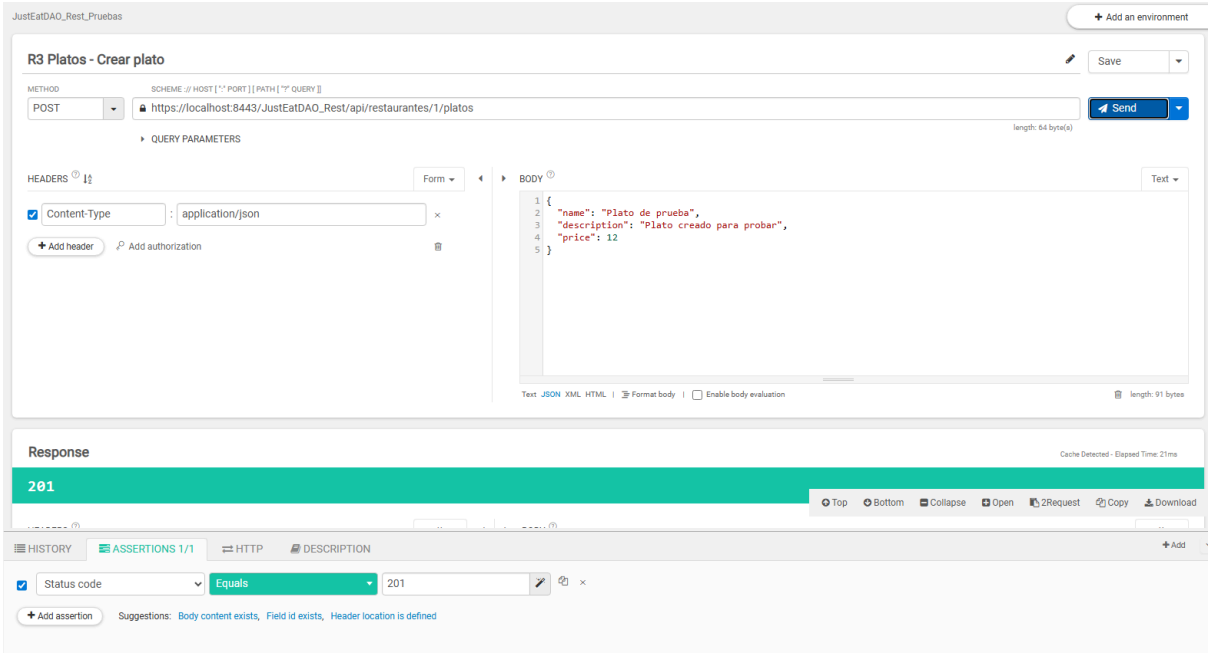
Endpoint:
/api/restaurantes/1/platos

Cuerpo:
{
"name": "Plato de prueba",
"description": "Plato creado para probar",
"price": 12
}

Objetivo:
Comprobar que el propietario de un restaurante puede crear platos para dicho restaurante.

Resultado esperado:
201 Created

Evidencia:



Prueba 3.3 - Intentar crear plato en restaurante ajeno

Usuario usado:
José Carlos Sancho

Email: unex@unex.es
ID: 1

Restaurante ajeno:
Burger King
ID: 0
Propietario: admin@admin.es

Plato creado:
Plato de prueba

Método:
POST

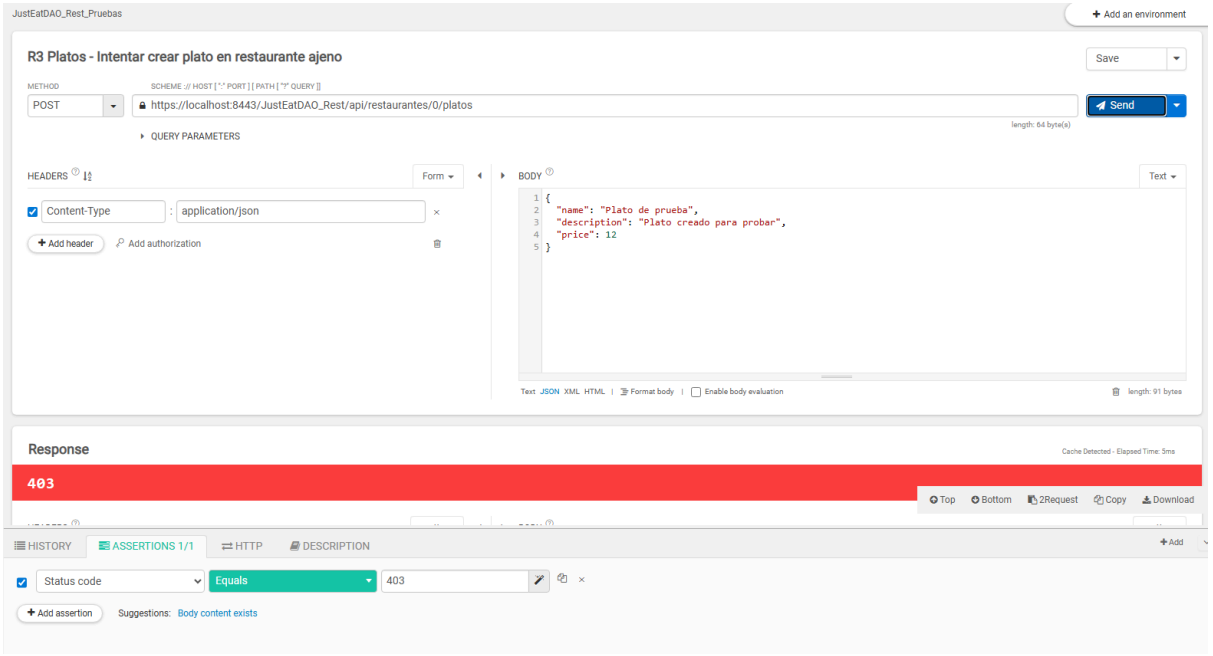
Endpoint:
/api/restaurantes/0/platos

Cuerpo:
```
{
  "name": "Plato de prueba",
  "description": "Plato creado para probar",
  "price": 12
}
```

Objetivo:
Comprobar que no se pueden crear platos en un restaurante ajeno

Resultado esperado:
403 Forbidden

Evidencia:



Prueba 3.4 - Editar plato creado

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

Plato editado:
Plato de prueba

Método:
PUT

Endpoint:
/api/platos/7

Nota:
Si el ID del plato creado no es 7, se sustituye por el ID real devuelto en la respuesta del POST.

Cuerpo:

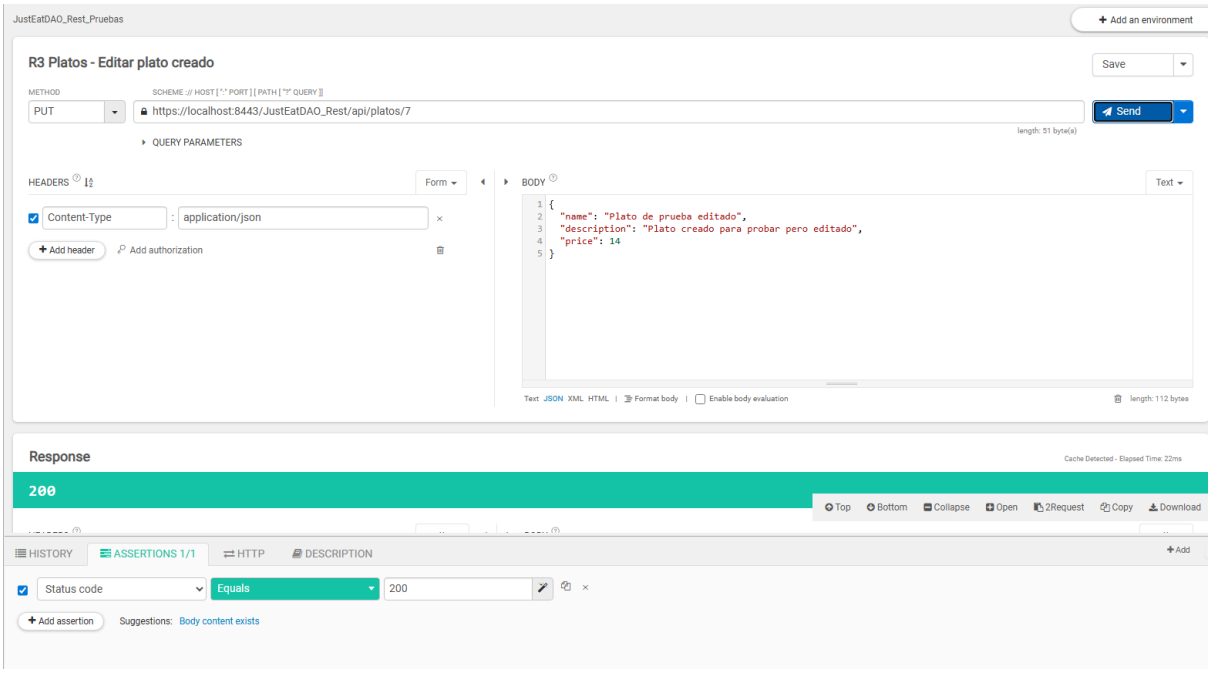
```
{
  "name": "Plato de prueba editado",
  "description": "Plato creado para probar pero editado",
  "price": 14
```

}

Objetivo:
Comprobar que el propietario puede editar un plato de su restaurante.

Resultado esperado:
200 OK

Evidencia:



Prueba 3.5 - Borrar plato creado

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

Plato borrado:
Plato de prueba editado

Método:
DELETE

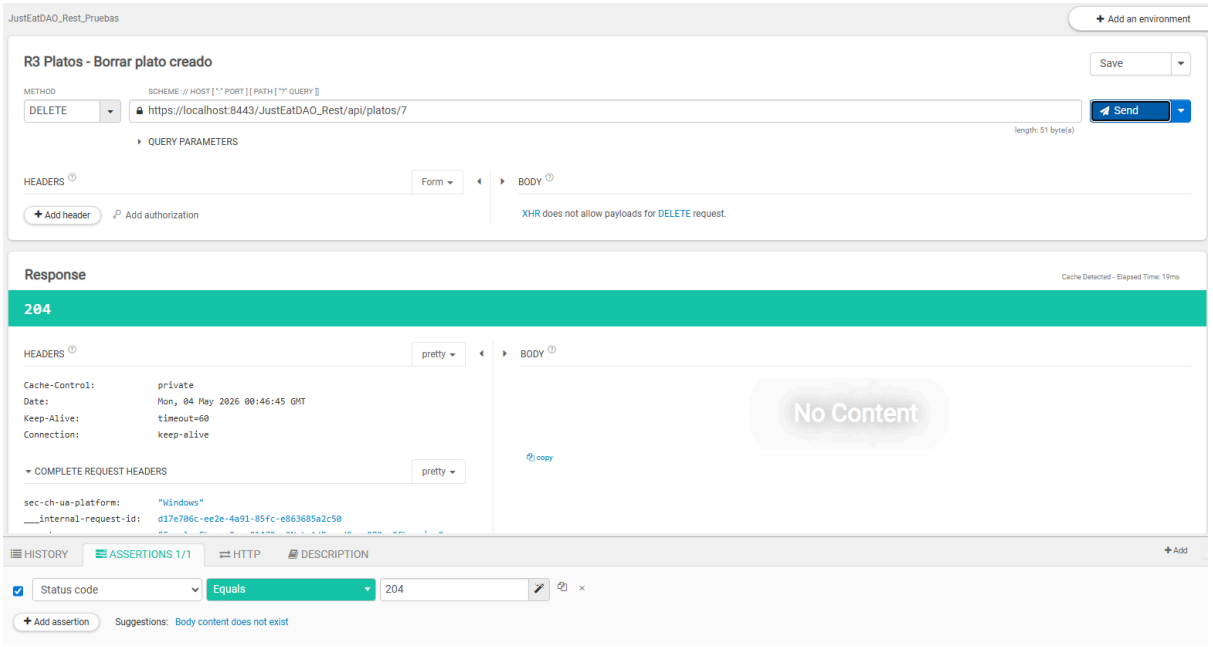
Endpoint:
/api/platos/7

Nota:
Si el ID del plato creado no es 6, se sustituye por el ID real devuelto en la respuesta del POST.

Objetivo:
Comprobar que el propietario puede eliminar un plato de su restaurante.

Resultado esperado:
204 No Content

Evidencia:



3.4 - Requisito 4. Gestión de pedidos

Llamadas a la API disponibles:

POST /api/pedidos
GET /api/usuarios/{id}/pedidos
GET /api/pedidos/{id}

Nota: También se tuvo que crear DTOs para gestionar los pedidos ya que no se podía pasar platos + restaurante en el mismo body.

Prueba 4.1 - Crear pedido

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

Método:
POST

Endpoint:
/api/pedidos

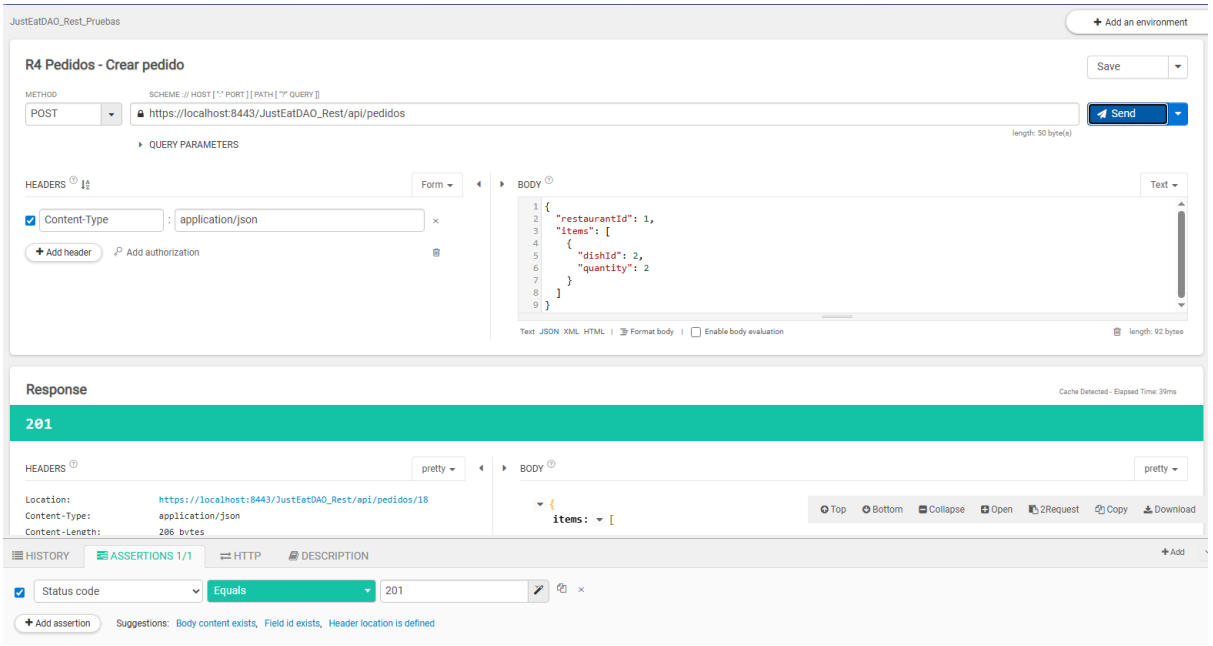
Cuerpo:

```
{
  "restaurantId": 1,
  "items": [
    {
      "dishId": 2,
      "quantity": 2
    }
  ]
}
```


Objetivo:
Comprobar que un usuario autenticado puede crear un pedido indicando el restaurante, los platos y la cantidad de cada plato.

Resultado esperado:
201 Created

Evidencia:



Prueba 4.2 - Consultar histórico de pedidos

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

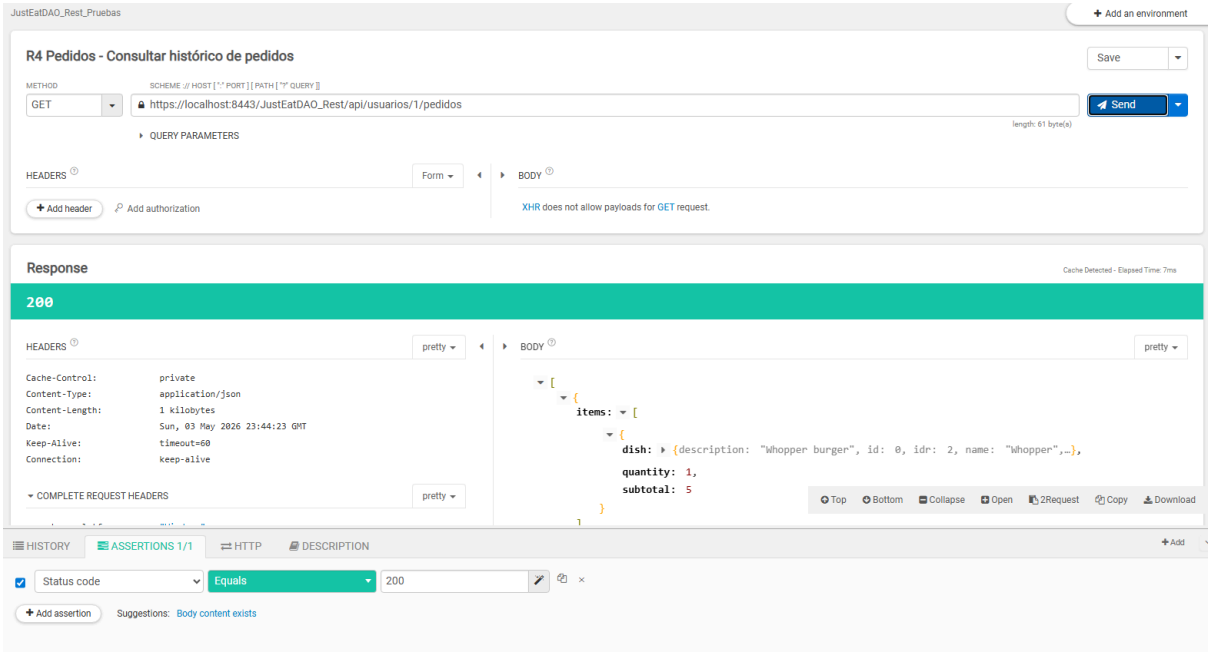
Método:
GET

Endpoint:
/api/usuarios/1/pedidos

Objetivo:
Comprobar que el usuario autenticado puede consultar su histórico de pedidos, incluyendo los platos y cantidades de cada pedido.

Resultado esperado:
200 OK

Evidencia:



Prueba 4.3 - Consultar pedido creado

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

Método:
GET

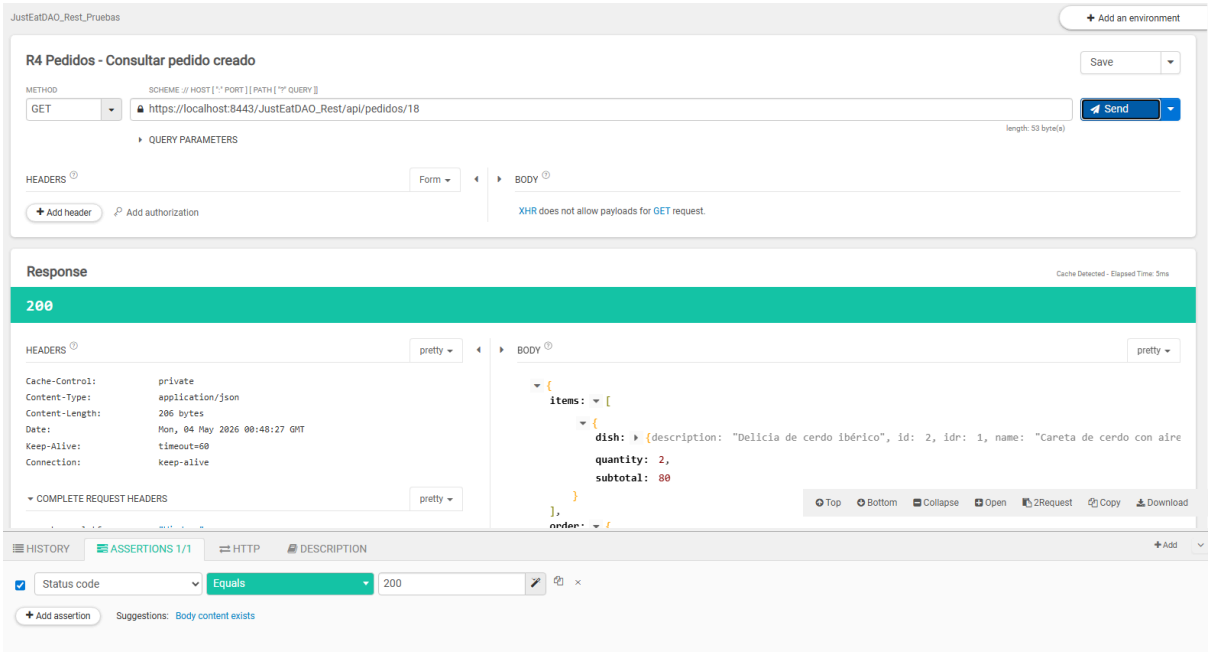
Endpoint:
/api/pedidos/18

Nota:
Si el ID del pedido creado no es 18, se sustituye por el ID real devuelto en la respuesta del POST.

Objetivo:
Comprobar que el usuario puede consultar el detalle de un pedido propio.

Resultado esperado:
200 OK

Evidencia:



3.5 - Requisito 5. Valoración

Llamadas a la API disponibles:

GET /api/restaurantes/{id}/valoraciones
POST /api/restaurantes/{id}/valoraciones
GET /api/restaurantes/all?sort=valoracionMedia

Prueba 5.1 - Consultar valoraciones de restaurante

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

Restaurante:
Burger King
ID: 0

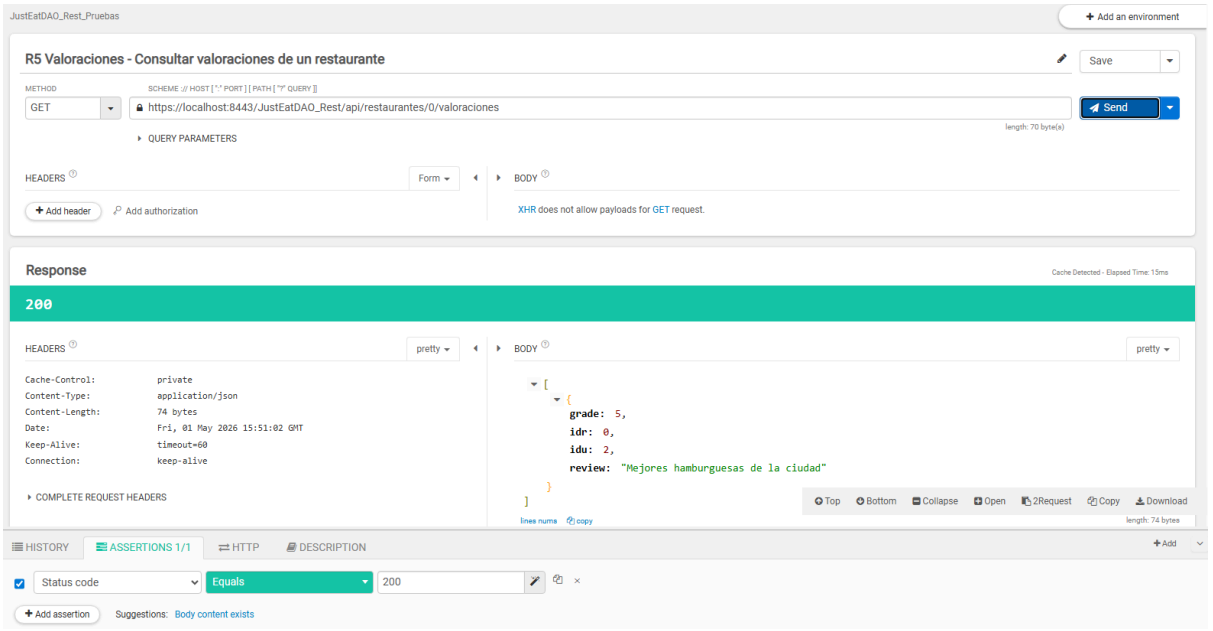
Método:
GET

Endpoint:
/api/restaurantes/0/valoraciones

Objetivo:
Comprobar que se pueden consultar las valoraciones de un restaurante.

Resultado esperado:
200 OK

Evidencia:



Prueba 5.2 - Crear o actualizar valoración

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

Restaurante:
Burger King
ID: 0

Método:
POST

Endpoint:
/api/restaurantes/0/valoraciones

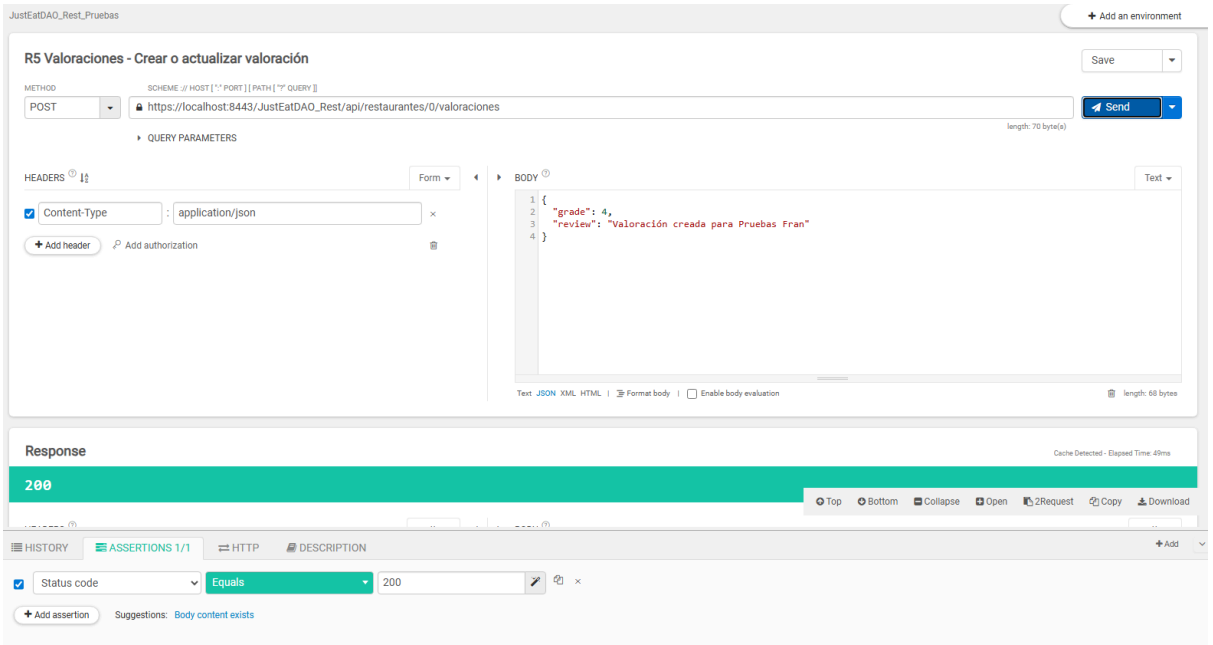
Cuerpo:
{

"grade": 4,
"review": "Valoración creada para Pruebas Fran"
}

Objetivo:
Comprobar que el usuario autenticado puede crear o actualizar su valoración de un restaurante y que no se crean valoraciones duplicadas para el mismo usuario y restaurante.

Resultado esperado:
200 OK

Evidencia:



Prueba 5.3 - Consultar restaurantes ordenados por valoración

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

Método:
GET

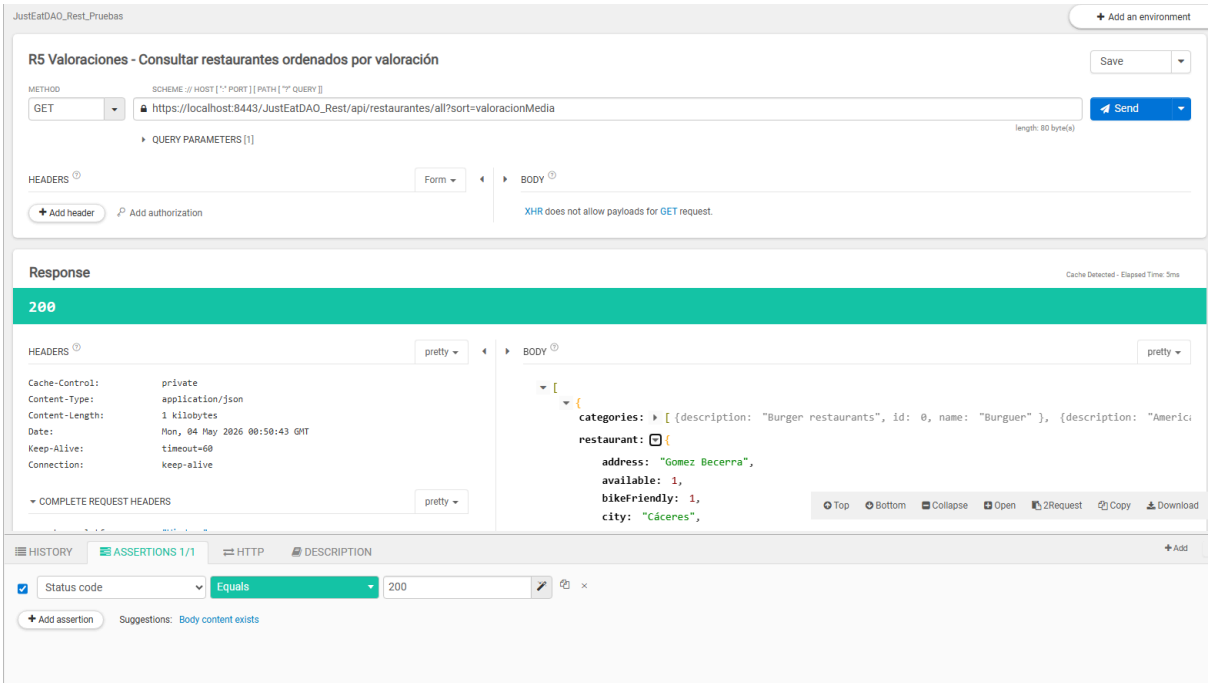
Endpoint:
/api/restaurantes/all?sort=valoracionMedia

Objetivo:

Comprobar que la API permite obtener los restaurantes ordenados por valoración media.

Resultado esperado:
200 OK

Evidencia:



3.6 - Requisito 6. Buscar restaurante

Llamadas a la API disponibles:

GET /api/restaurantes/all?q={texto}
GET /api/restaurantes/all?localidad={ciudad}

Prueba 6.1 - Buscar por título del restaurante

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

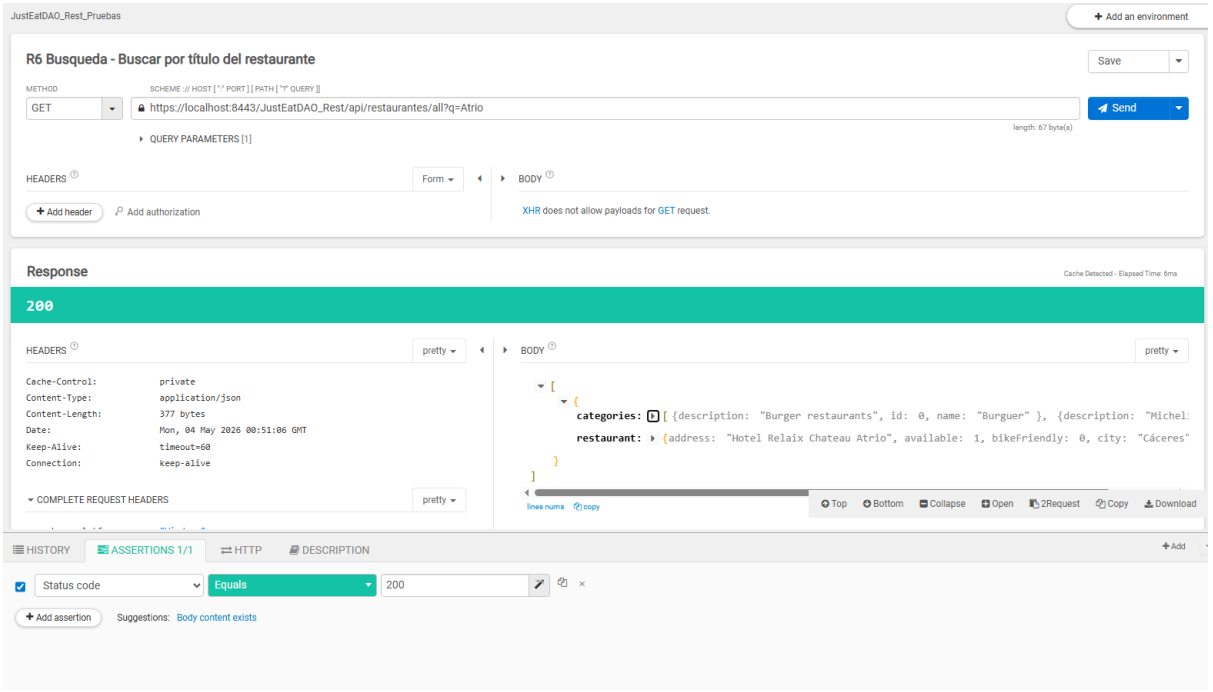
Método:
GET

Endpoint:
/api/restaurantes/all?q=Atrio

Objetivo:
Comprobar que se pueden buscar restaurantes por su título, en este caso el nombre Atrio.

Resultado esperado:
200 OK

Evidencia:



Prueba 6.2 - Buscar restaurantes por localidad

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

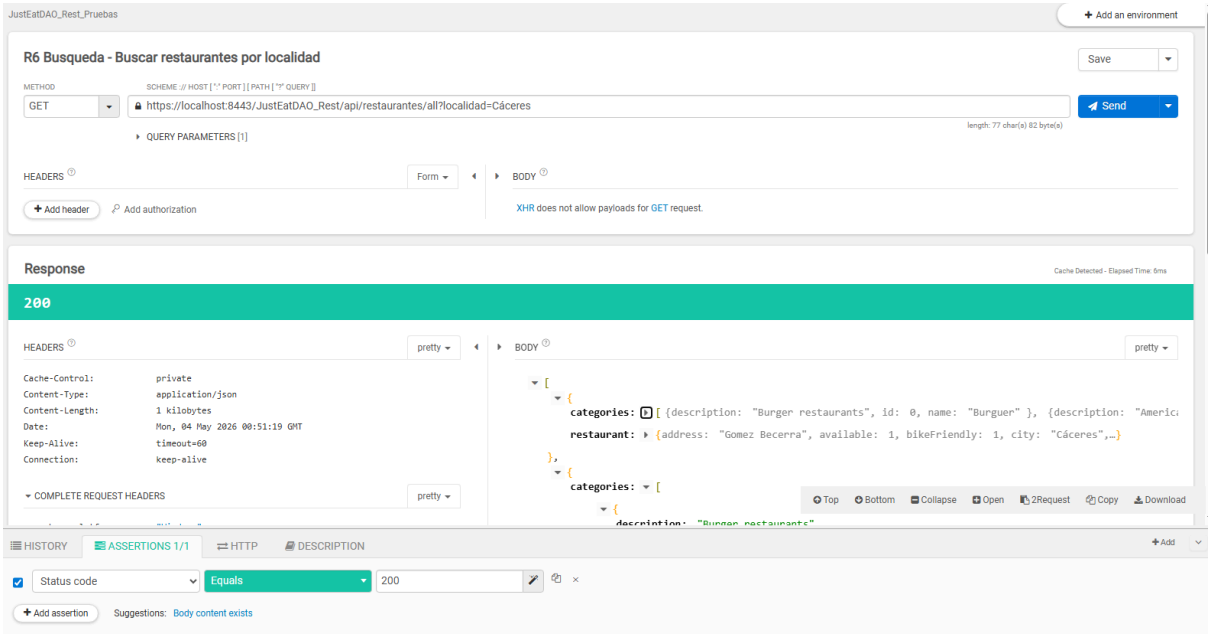
Método:
GET

Endpoint:
/api/restaurantes/all?localidad=Cáceres

Objetivo:
Comprobar que se pueden filtrar restaurantes por localidad.

Resultado esperado:
200 OK

Evidencia:



3.7 - Requisito 7. Estado del restaurante

Llamadas a la API disponibles:

PUT /api/restaurantes/{id}/estado
GET /api/restaurantes/all?estado={estado}

Prueba 7.1 - Cambiar estado de restaurante

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

Restaurante:
Atrio

Método:

PUT

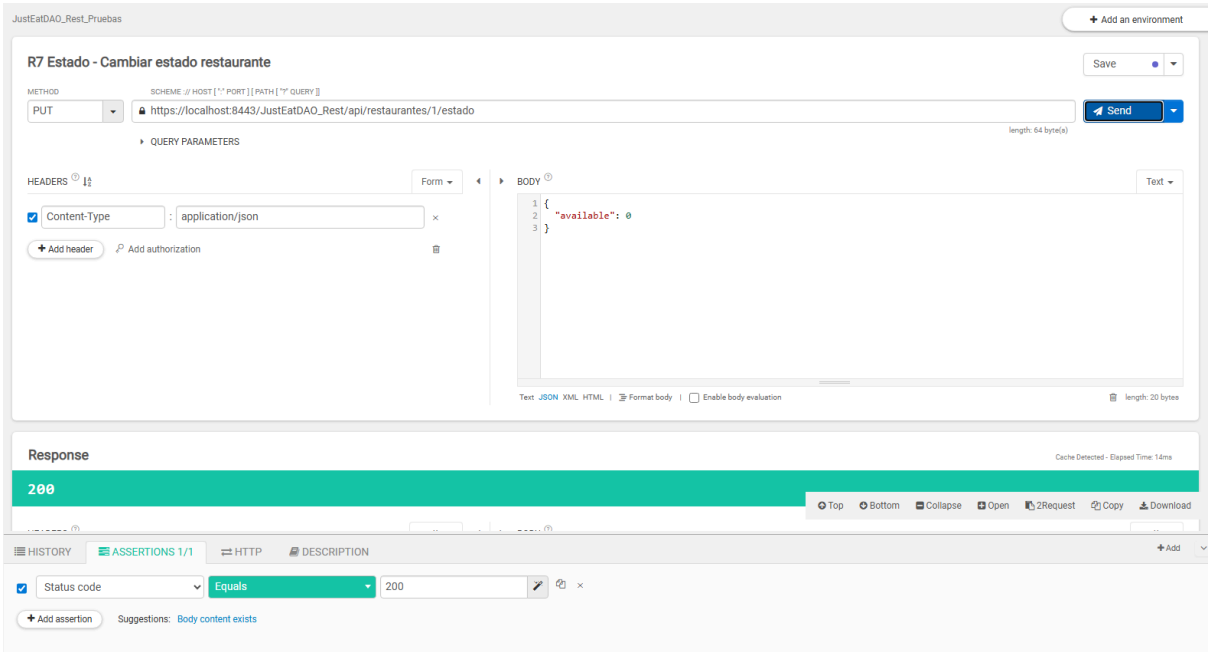
Endpoint:
/api/restaurantes/1/estado

Cuerpo:
{
"available": 0
}

Objetivo:
Comprobar que el propietario puede cambiar el estado de su restaurante a no disponible o no acepta pedidos.

Resultado esperado:
200 OK

Evidencia:



Prueba 7.2 - Intentar cambiar estado de restaurante ajeno

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

Restaurante ajeno:

Burger King
ID: 0
Propietario: admin@admin.es

Método:
PUT

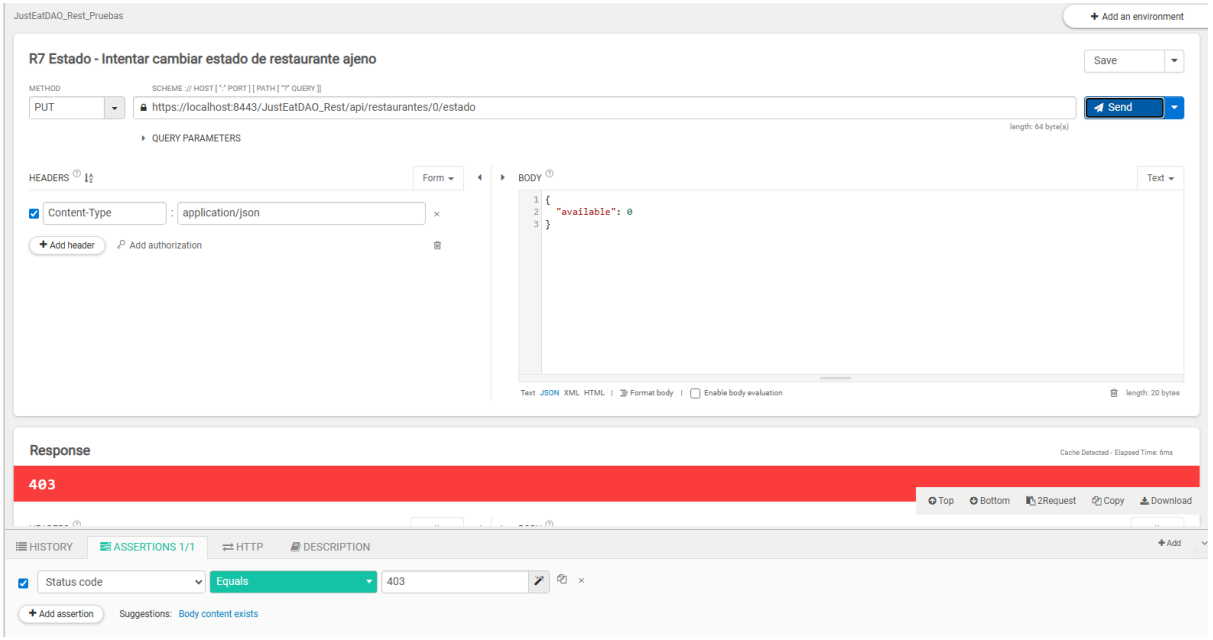
Endpoint:
/api/restaurantes/0/estado

Cuerpo:
```
{
  "available": 0
}
```

Objetivo:
Comprobar que no se puede cambiar el estado de un restaurante ajeno.

Resultado esperado:
403 Forbidden

Evidencia:



Prueba 7.3 - Filtrar restaurantes que aceptan pedidos

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

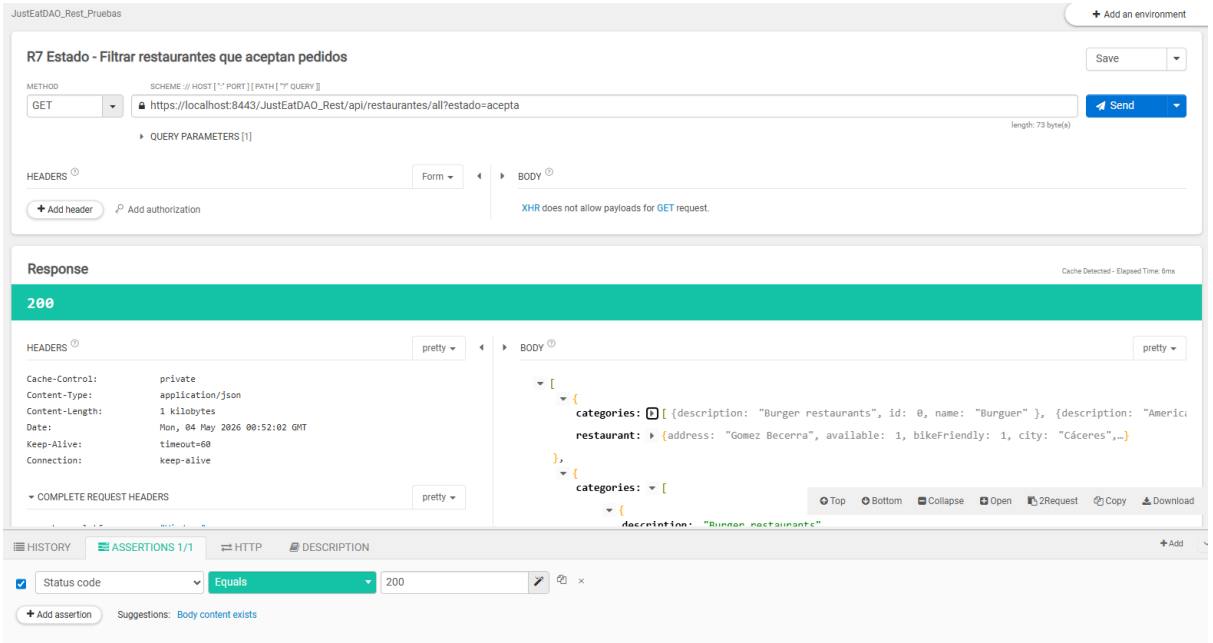
Método:
GET

Endpoint:
/api/restaurantes/all?estado=acepta

Objetivo:
Comprobar que la API permite filtrar restaurantes para mostrar únicamente los que aceptan pedidos.

Resultado esperado:
200 OK

Evidencia:



4 - Requisitos extra

4.0 - Requisito Extra. Inicio de sesión REST

Llamadas a la API disponibles:

POST /api/sesiones

Prueba RE0.1 - Iniciar sesión REST

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

Email:
unex@unex.es

Método:
POST

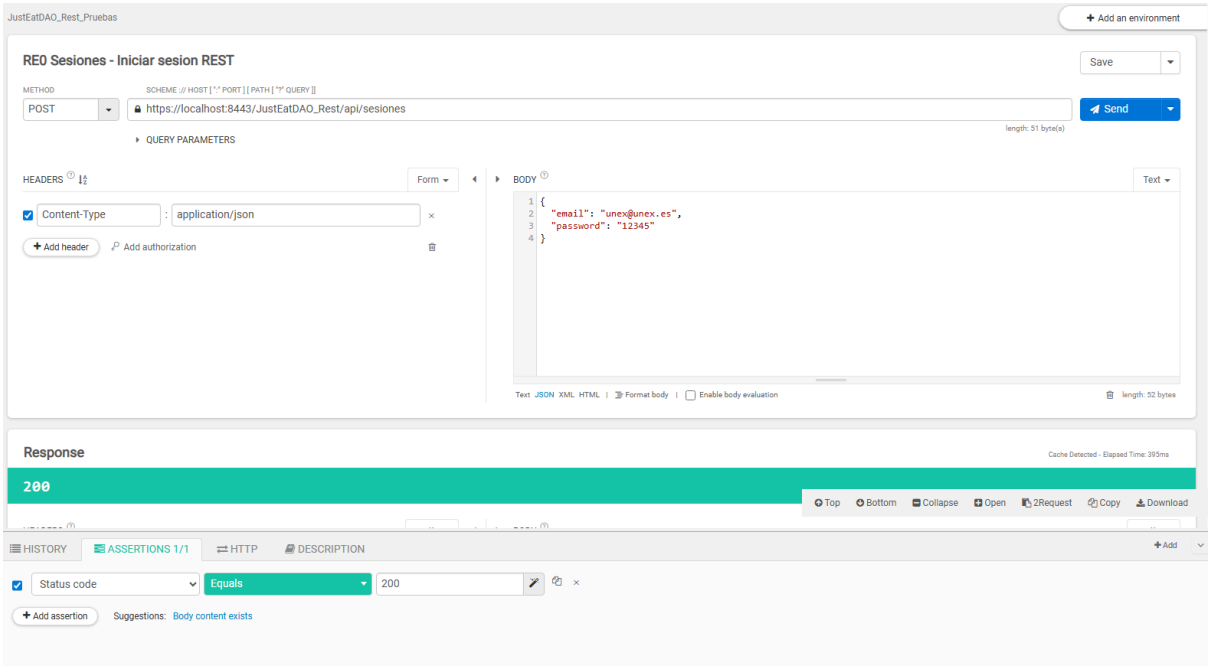
Endpoint:
/api/sesiones

Cuerpo:
{
"email": "unex@unex.es",
"password": "12345"
}

Objetivo:
Comprobar que se puede iniciar sesión desde la API REST usando email y contraseña.

Resultado esperado:
200 OK

Evidencia:



4.1 - Requisito Extra 1. Gestión de categorías

Llamadas a la API disponibles:

GET /api/categorias
GET /api/categorias/{id}
POST /api/categorias
PUT /api/categorias/{id}
DELETE /api/categorias/{id}
GET /api/restaurantes/{id}/categorias
POST /api/restaurantes/{id}/categorias/{idCategoria}
DELETE /api/restaurantes/{id}/categorias/{idCategoria}
GET /api/categorias/{id}/restaurantes

Prueba RE1.1 - Listar categorías

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

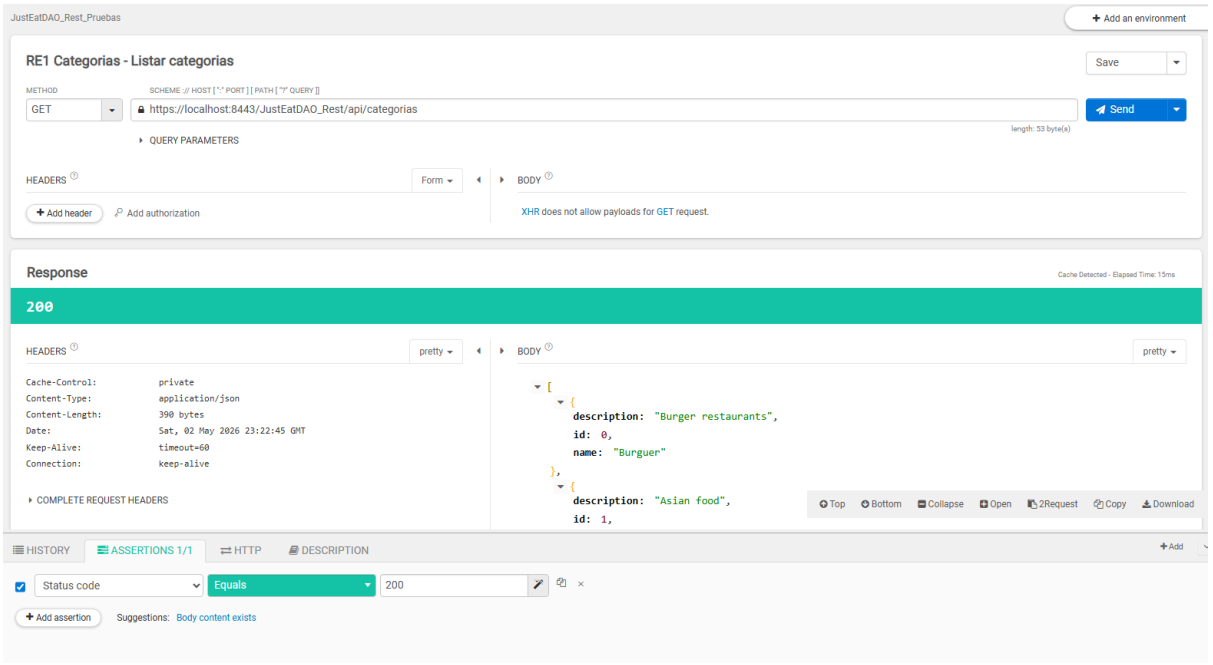
Método:
GET

Endpoint:
/api/categorias

Objetivo:
Comprobar que la API devuelve el listado de categorías disponibles.

Resultado esperado:
200 OK

Evidencia:



Prueba RE1.2 - Consultar categoría

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

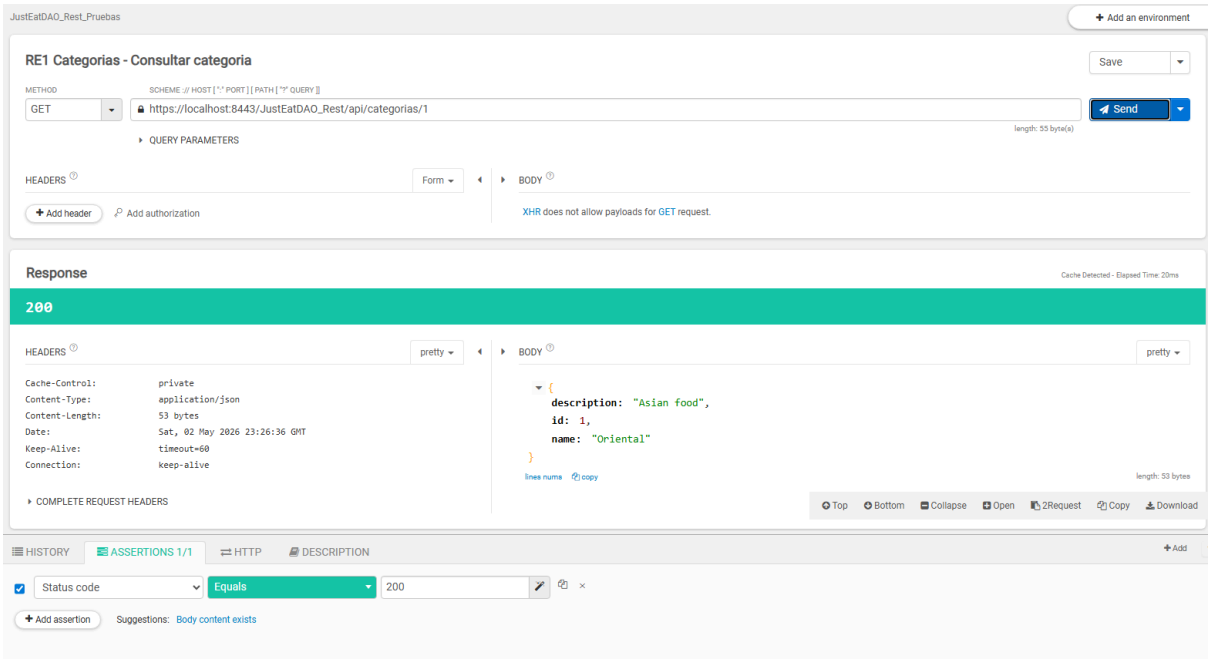
Método:
GET

Endpoint:
/api/categorias/1

Objetivo:
Comprobar que se puede consultar una categoría concreta existente.

Resultado esperado:
200 OK

Evidencia:



Prueba RE1.3 - Crear categoría de prueba

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

Método:
POST

Endpoint:
/api/categorias

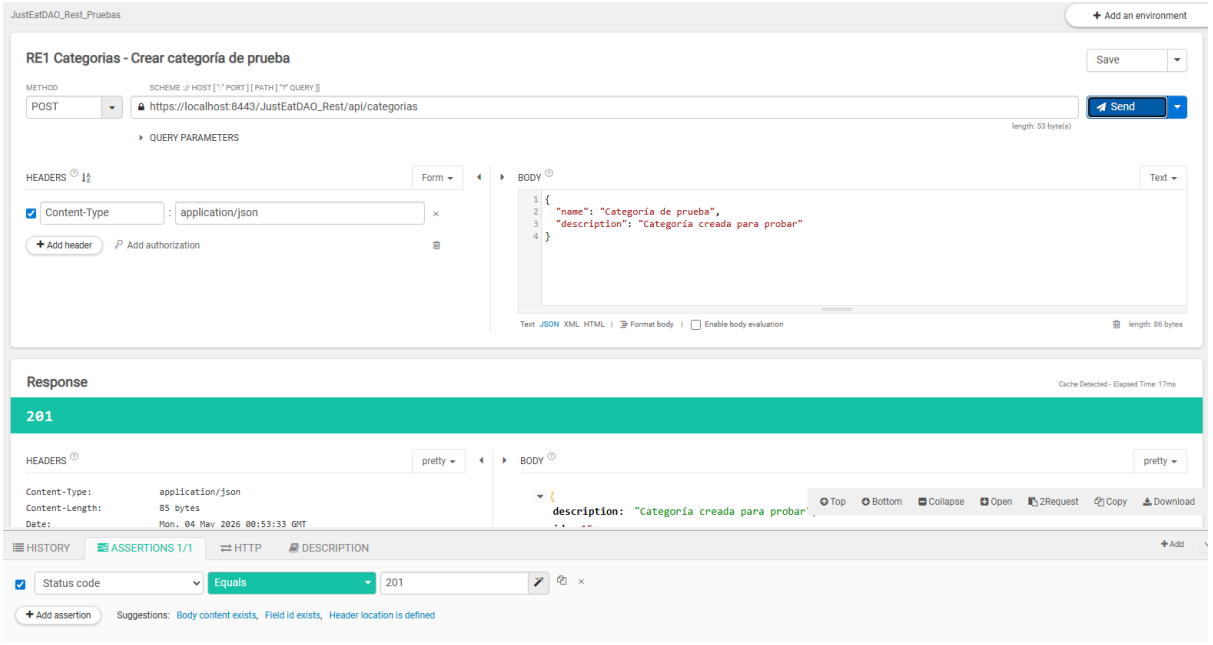
Cuerpo:
{
"name": "Categoría de prueba",
"description": "Categoría creada para probar"
}

Objetivo:
Comprobar que se puede crear una categoría nueva.

Resultado esperado:

201 Created

Evidencia:



Prueba RE1.4 - Editar categoría de prueba

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

Método:
PUT

Endpoint:
/api/categorias/15

Cuerpo:
{
"name": "Categoría de prueba editada",
"description": "Categoría editada creada para probar"
}

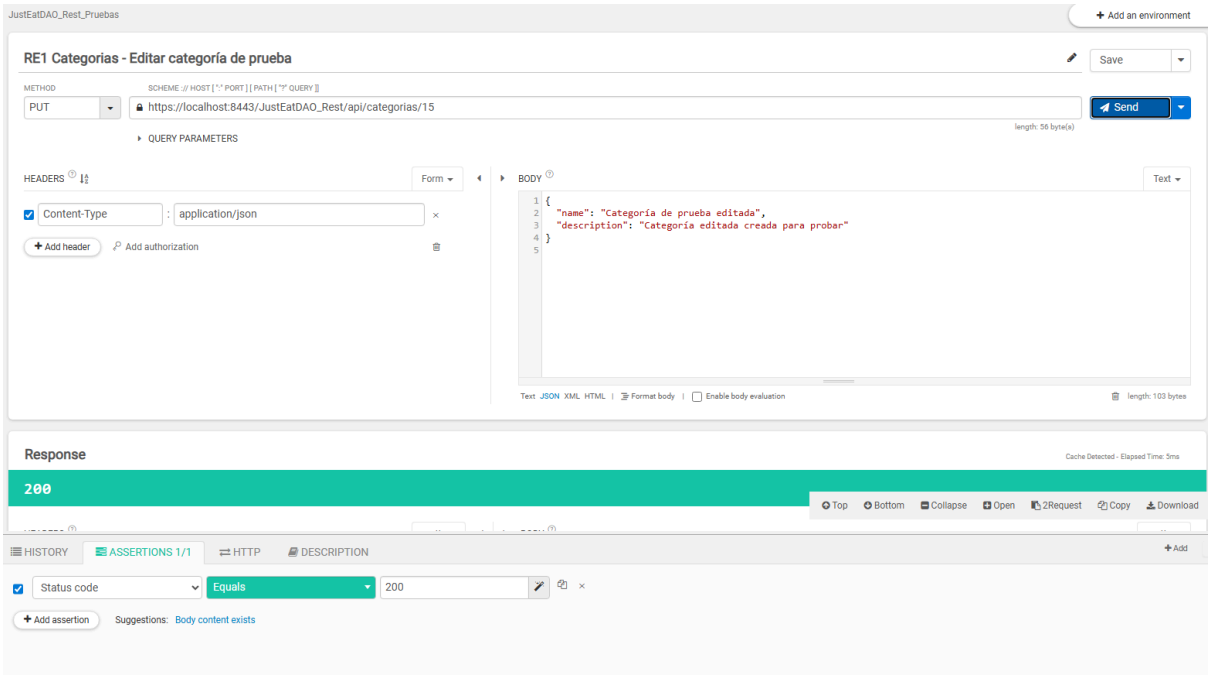
Nota:
Si el ID devuelto al crear la categoría no es 15, se sustituye por el ID real.

Objetivo:

Comprobar que se puede editar una categoría existente.

Resultado esperado:
200 OK

Evidencia:



Prueba RE1.5 - Agregar categoría a restaurante propio

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

Método:
POST

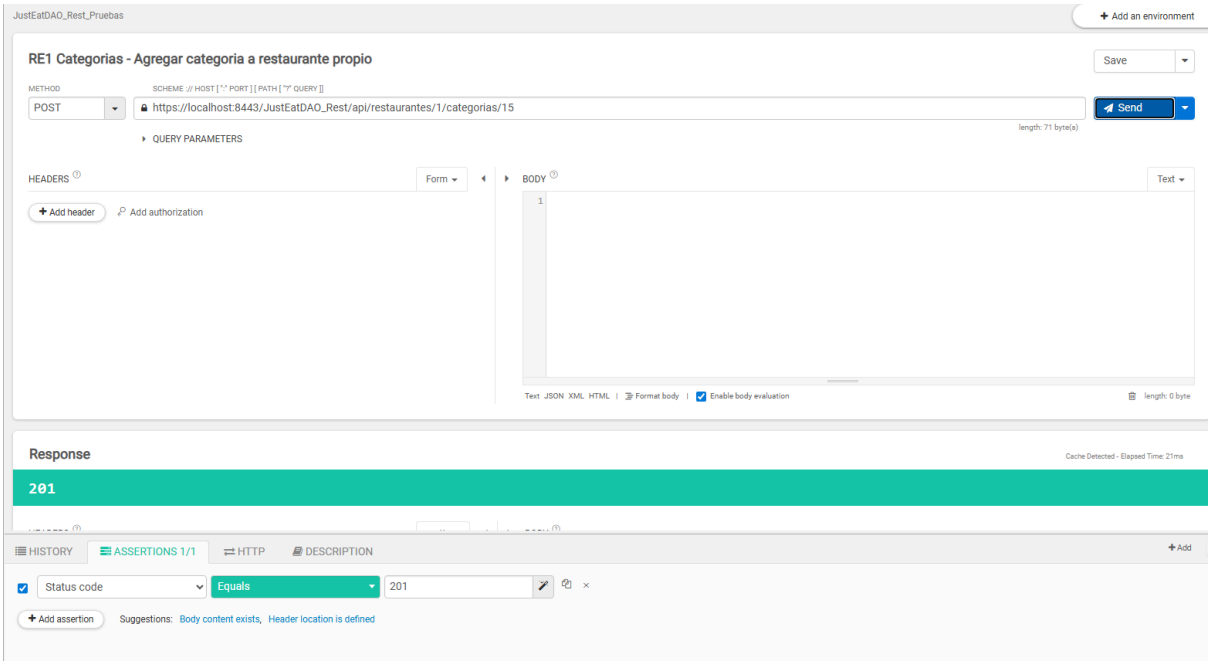
Endpoint:
/api/restaurantes/1/categorias/15

Nota:
Si el ID devuelto al crear la categoría no es 15, se sustituye 15 por el ID real devuelto en la respuesta del POST de creación de categoría.

Objetivo:
Comprobar que el propietario puede agregar una categoría a su restaurante.

Resultado esperado:
201 Created

Evidencia:



Prueba RE1.6 - Listar categorías de restaurante

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

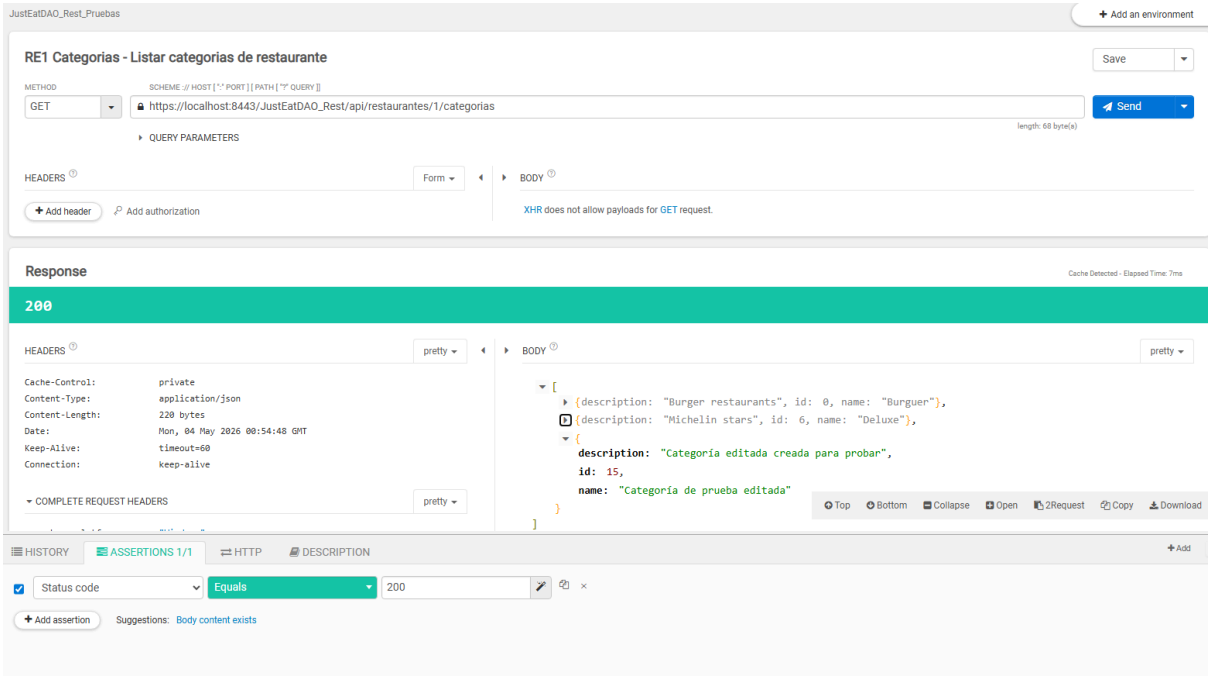
Método:
GET

Endpoint:
/api/restaurantes/1/categorias

Objetivo:
Comprobar que se pueden consultar las categorías asociadas a un restaurante.

Resultado esperado:
200 OK

Evidencia:



Prueba RE1.7 - Intentar asociar categoría a restaurante ajeno

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

Método:
POST

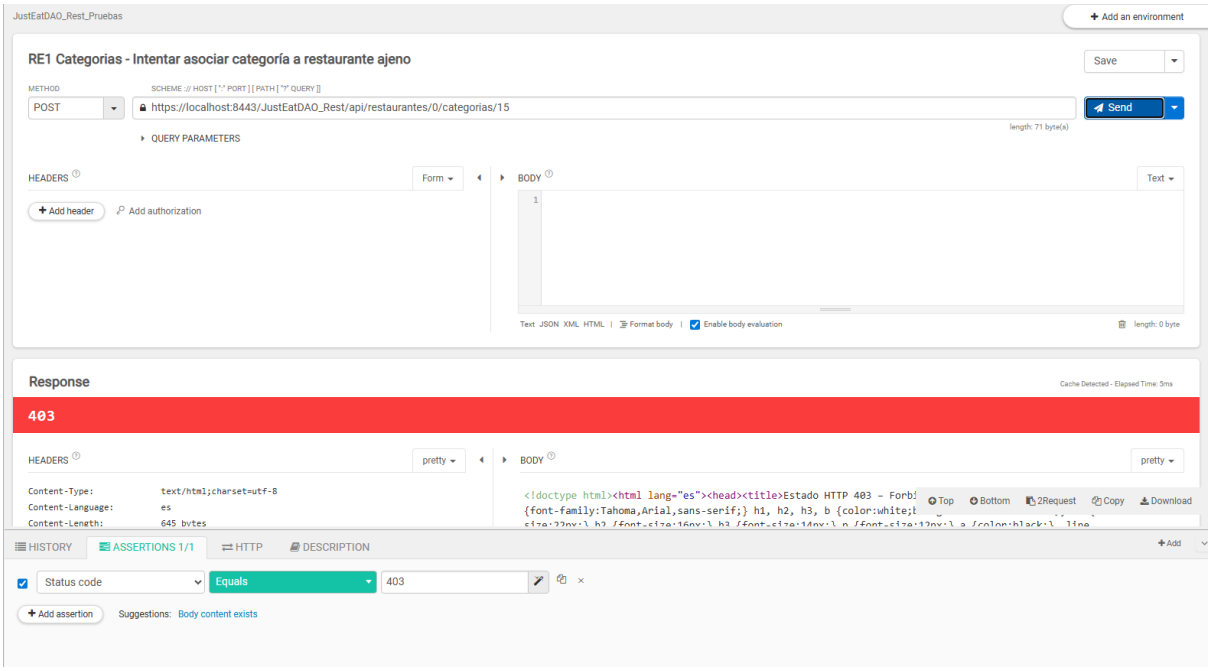
Endpoint:
/api/restaurantes/0/categorias/15

Nota:
Si el ID devuelto al crear la categoría no es 15, se sustituye 15 por el ID real devuelto en la respuesta del POST de creación de categoría.

Objetivo:
Comprobar que un usuario no puede modificar categorías de un restaurante que no es suyo.

Resultado esperado:
403 Forbidden

Evidencia:



Prueba RE1.8 - Consultar restaurantes por categoría

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

Método:
GET

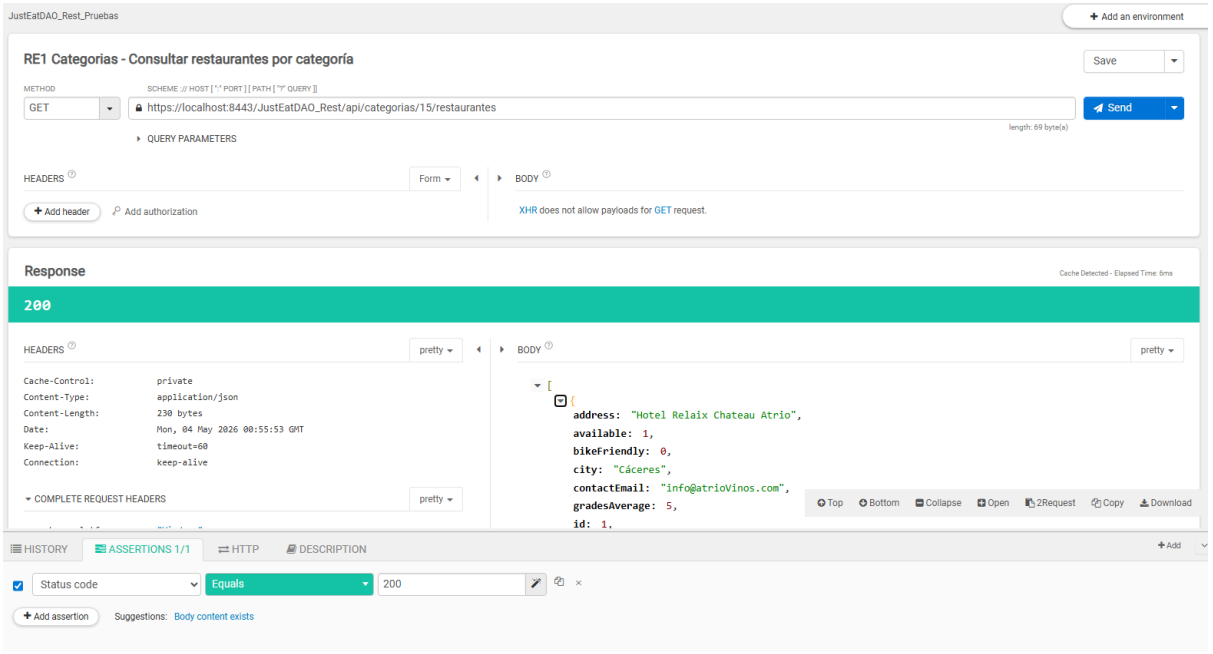
Endpoint:
/api/categorias/7/restaurantes

Nota:
Si se quiere consultar la categoría creada durante las pruebas, se sustituye el ID de la URL por el ID real devuelto en el POST de creación de categoría.

Objetivo:
Comprobar el acceso rápido a restaurantes asociados a una categoría.

Resultado esperado:
200 OK

Evidencia:



Prueba RE1.9 - Quitar categoría de restaurante

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

Método:
DELETE

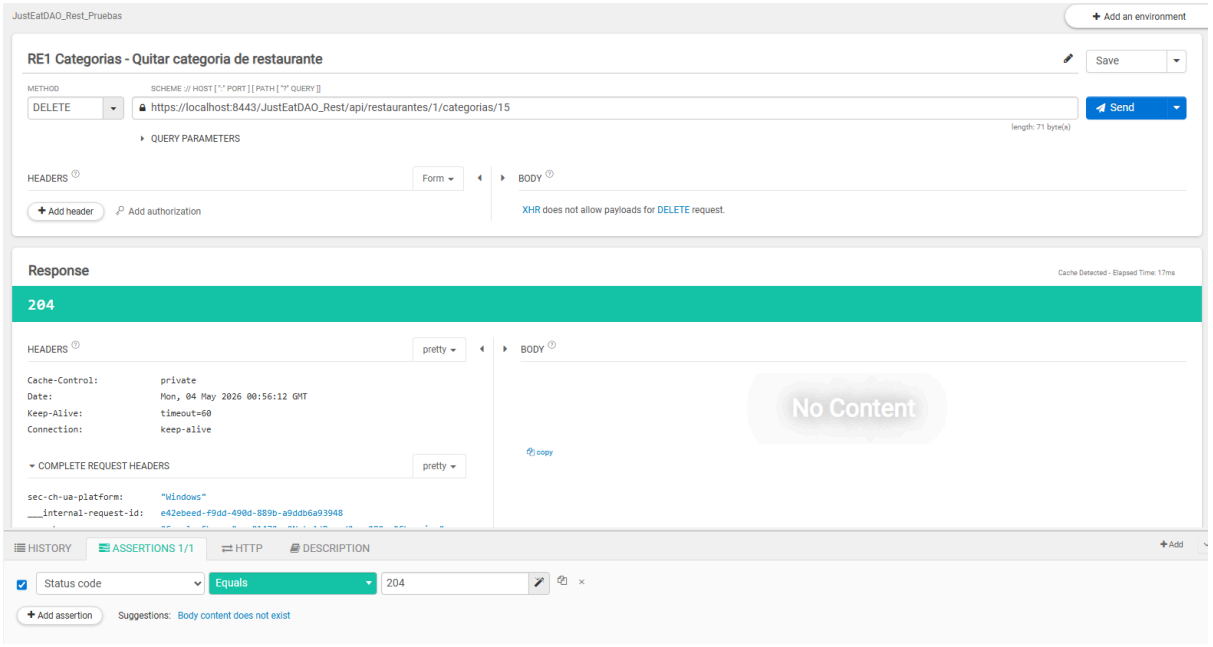
Endpoint:
/api/restaurantes/1/categorias/13

Nota:
Si el ID devuelto al crear la categoría no es 13, se sustituye 13 por el ID real devuelto en la respuesta del POST de creación de categoría.

Objetivo:
Comprobar que el propietario puede eliminar una categoría asociada a su restaurante.

Resultado esperado:
204 No Content

Evidencia:



Prueba RE1.10 - Borrar categoría de prueba

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

Método:
DELETE

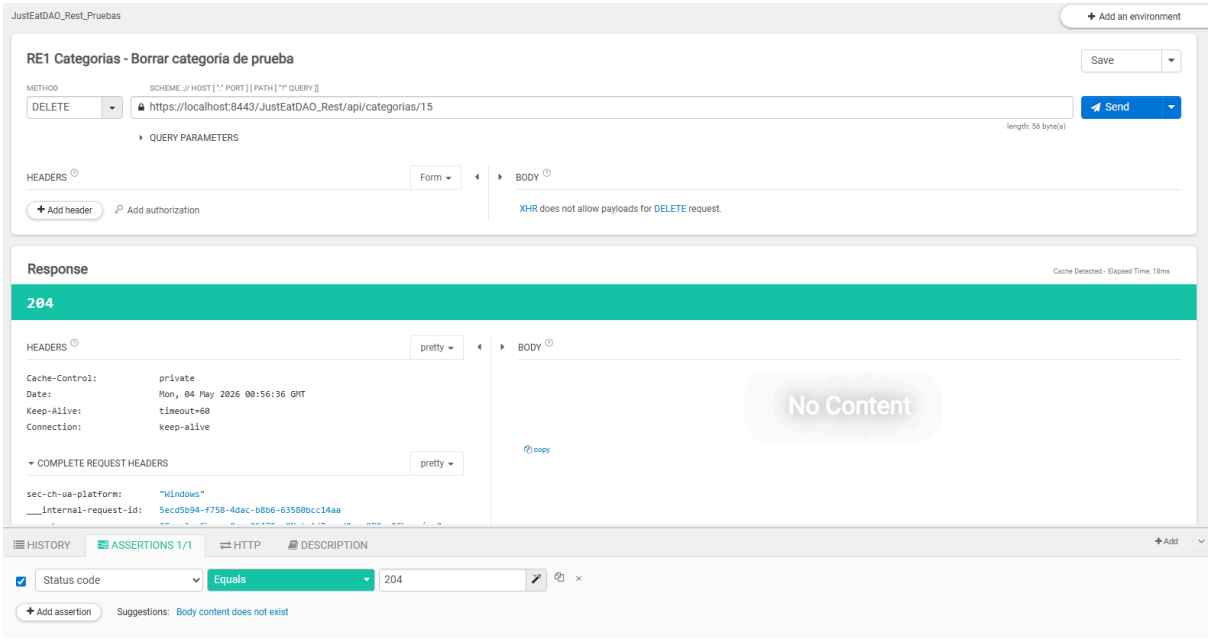
Endpoint:
/api/categorias/15

Nota:
Si el ID devuelto al crear la categoría no es 15, se sustituye 15 por el ID real devuelto en la respuesta del POST de creación de categoría.

Objetivo:
Comprobar que se puede eliminar la categoría temporal creada para las pruebas.

Resultado esperado:
204 No Content

Evidencia:



4.2 - Requisito Extra 2. Restaurantes favoritos

Llamadas a la API disponibles:

GET /api/usuarios/{id}/favoritos
POST /api/restaurantes/{id}/favoritos
DELETE /api/restaurantes/{id}/favoritos

Prueba RE2.1 - Añadir restaurante a favoritos

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

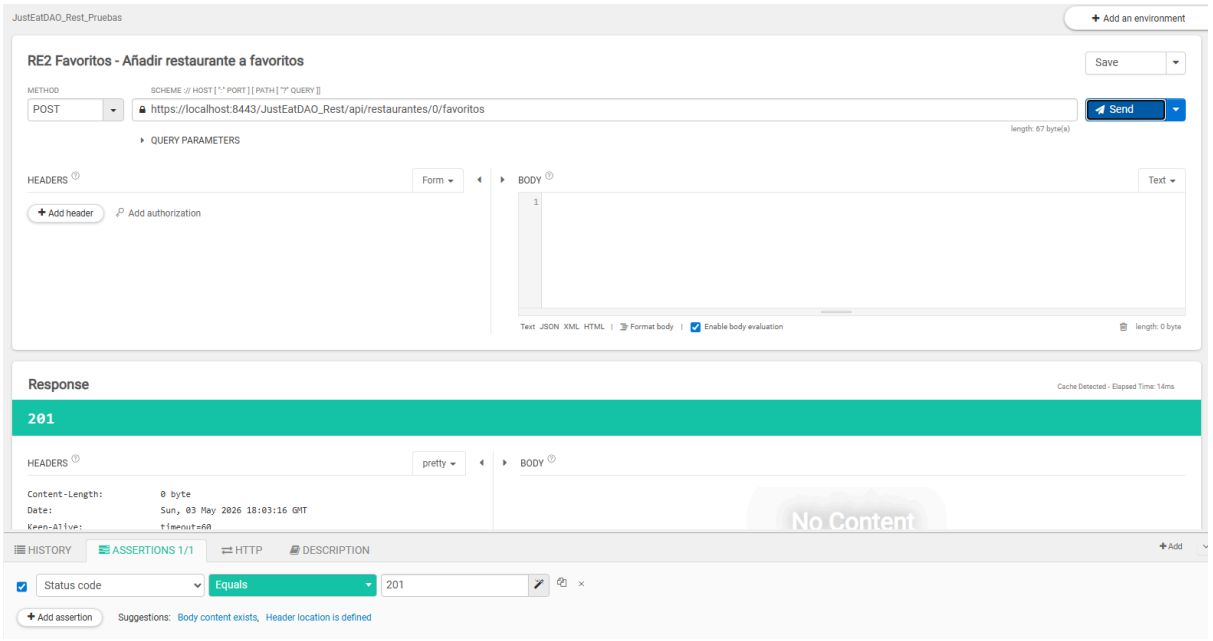
Método:
POST

Endpoint:
/api/restaurantes/0/favoritos

Objetivo:
Comprobar que el usuario puede añadir un restaurante a favoritos.

Resultado esperado:
201 Created

Evidencia:



Prueba RE2.2 - Listar favoritos

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

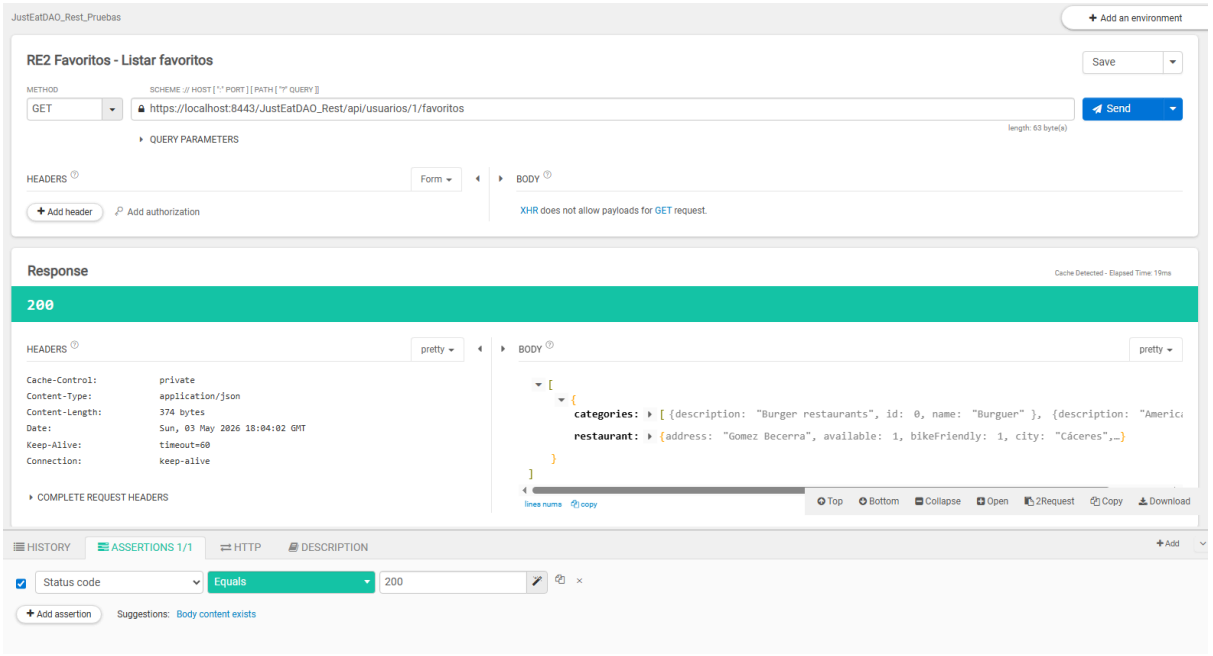
Método:
GET

Endpoint:
/api/usuarios/1/favoritos

Objetivo:
Comprobar que el usuario puede consultar sus restaurantes favoritos.

Resultado esperado:
200 OK

Evidencia:



Prueba RE2.3 - Intentar añadir favorito duplicado

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

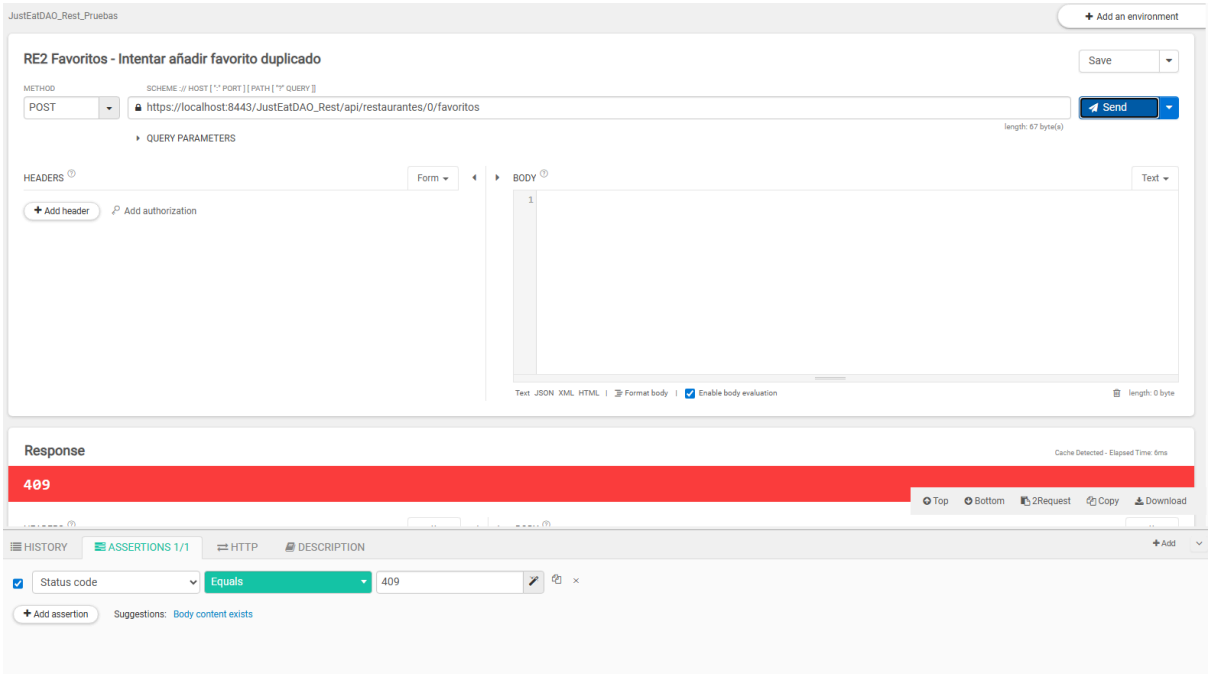
Método:
POST

Endpoint:
/api/restaurantes/0/favoritos

Objetivo:
Comprobar que no se puede añadir dos veces el mismo restaurante a favoritos.

Resultado esperado:
409 Conflict

Evidencia:



Prueba RE2.4 - Consultar favoritos de usuario ajeno

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

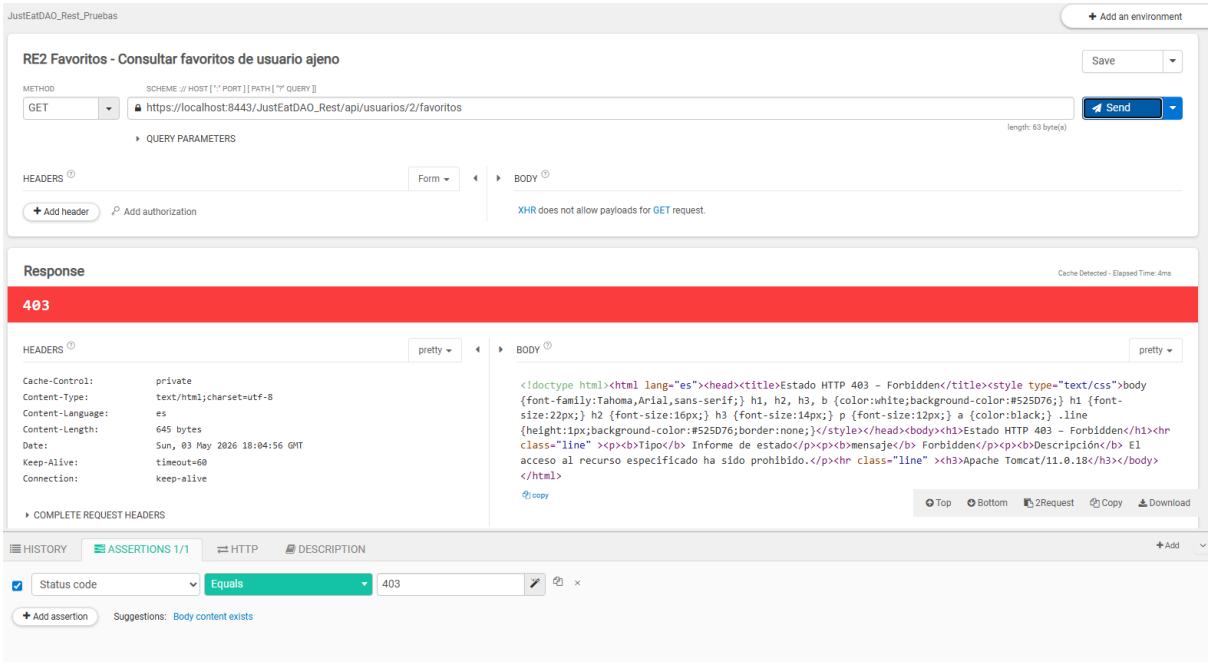
Método:
GET

Endpoint:
/api/usuarios/2/favoritos

Objetivo:
Comprobar que un usuario no puede consultar los favoritos privados de otro usuario.

Resultado esperado:
403 Forbidden

Evidencia:



Prueba RE2.5 - Quitar restaurante de favoritos

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

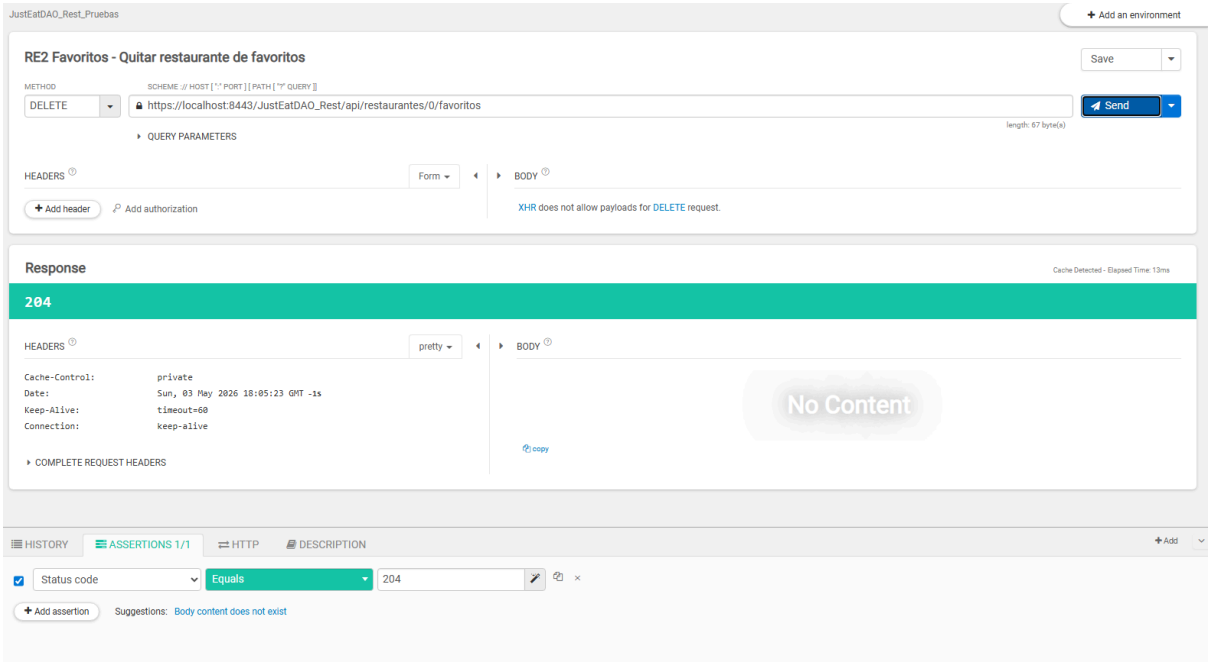
Método:
DELETE

Endpoint:
/api/restaurantes/0/favoritos

Objetivo:
Comprobar que el usuario puede quitar un restaurante de favoritos.

Resultado esperado:
204 No Content

Evidencia:



4.3 - Requisito Extra 3. Búsqueda avanzada

Llamadas a la API disponibles:

GET /api/restaurantes/all?categoria={id}
GET /api/restaurantes/all?comentarios={texto}
GET /api/restaurantes/all?plato={texto}
GET /api/restaurantes/all?minPrecio={min}&maxPrecio={max}
GET /api/restaurantes/all con varios filtros combinados

Prueba RE3.1 - Filtrar por categoría

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

Categoría a filtrar:
Nombre: Oriental
ID: 1

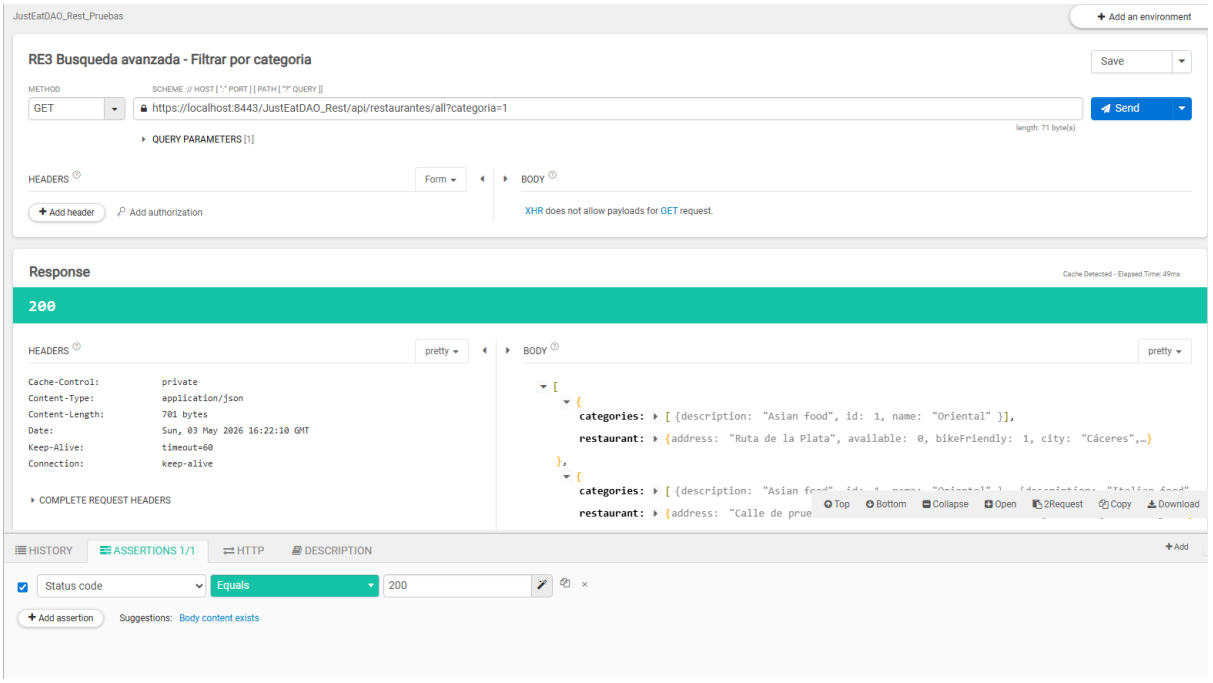
Método:
GET

Endpoint:
/api/restaurantes/all?categoria=1

Objetivo:
Comprobar que se pueden filtrar restaurantes por categoría.

Resultado esperado:
200 OK

Evidencia:



Prueba RE3.2 - Filtrar por comentarios

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

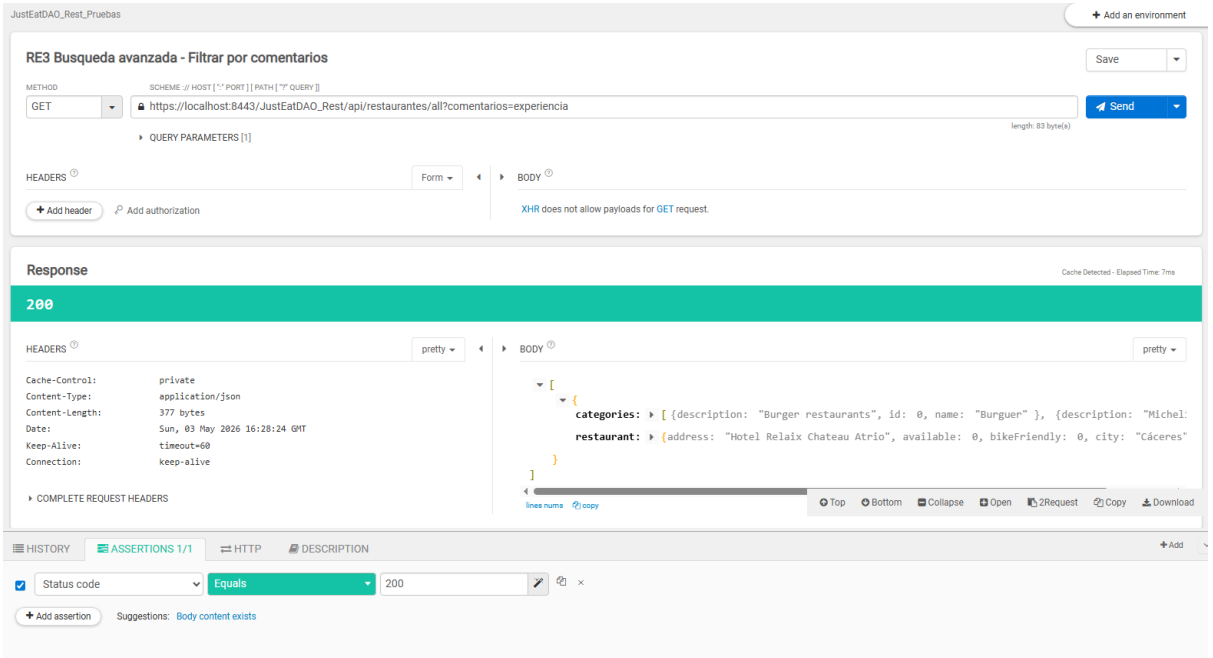
Método:
GET

Endpoint:
/api/restaurantes/all?comentarios=experiencia

Objetivo:
Comprobar que se pueden buscar restaurantes por texto en comentarios.

Resultado esperado:
200 OK

Evidencia:



Prueba RE3.3 - Filtrar por plato

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

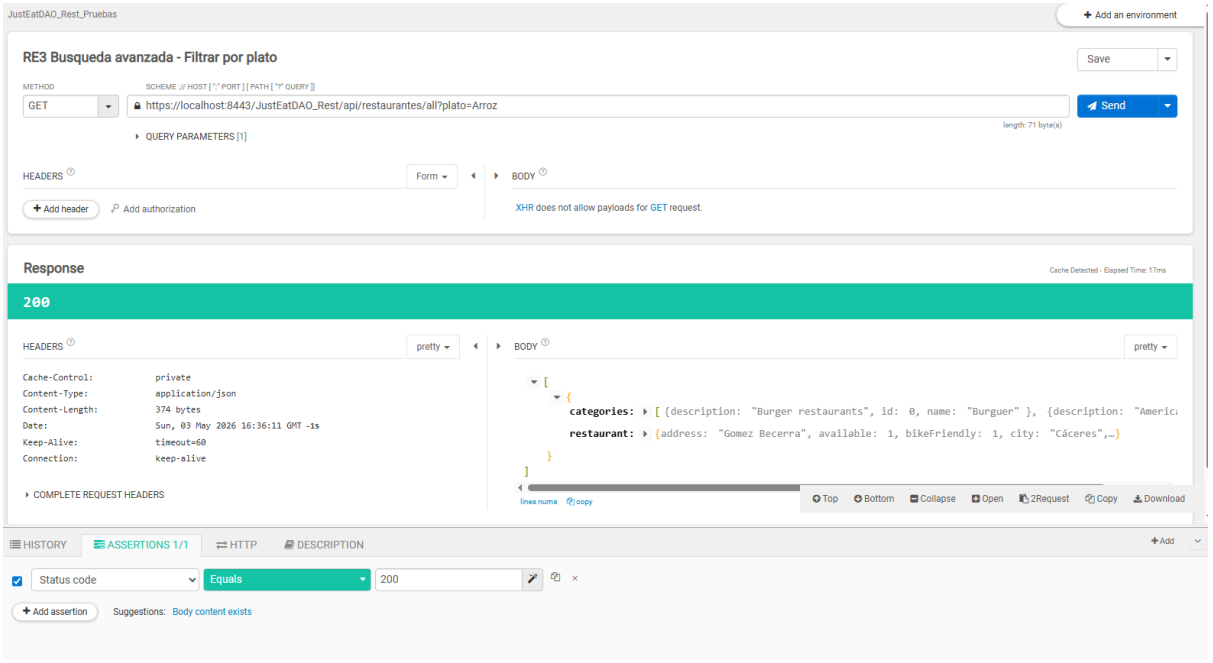
Método:
GET

Endpoint:
/api/restaurantes/all?plato=Arroz

Objetivo:
Comprobar que se pueden buscar restaurantes que ofrecen un plato concreto.

Resultado esperado:
200 OK

Evidencia:



Prueba RE3.4 - Filtrar por rango de precios

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

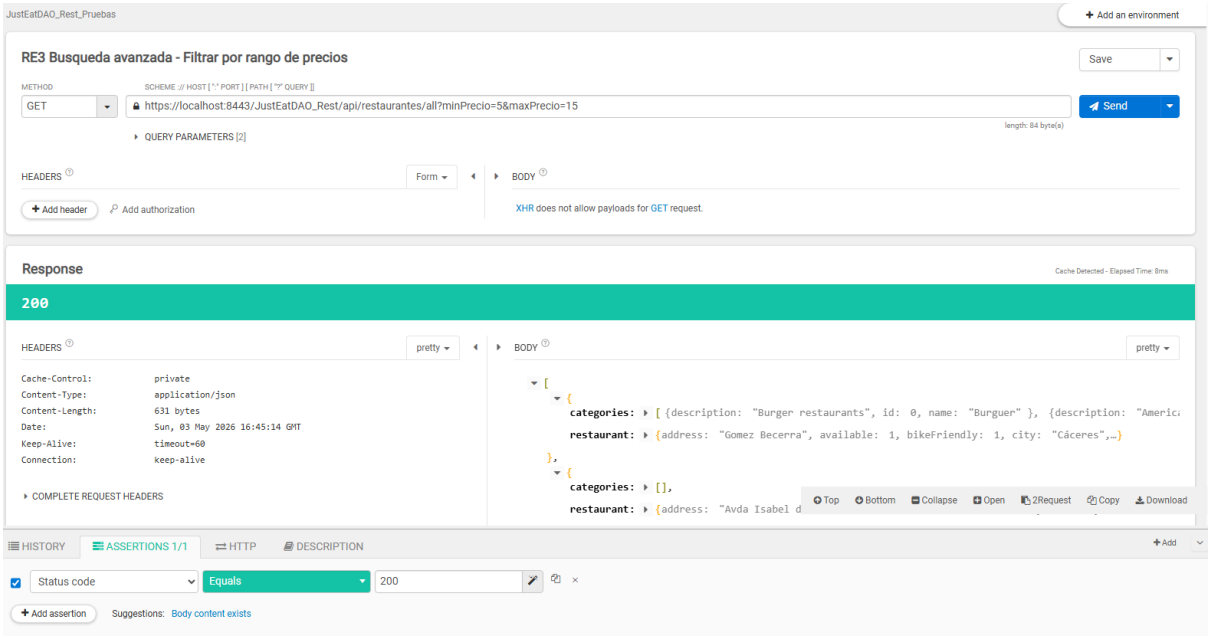
Método:
GET

Endpoint:
/api/restaurantes/all?minPrecio=5&maxPrecio=15

Objetivo:
Comprobar que se pueden filtrar restaurantes por rango de precios.

Resultado esperado:
200 OK

Evidencia:



Prueba RE3.5 - Filtros combinados

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

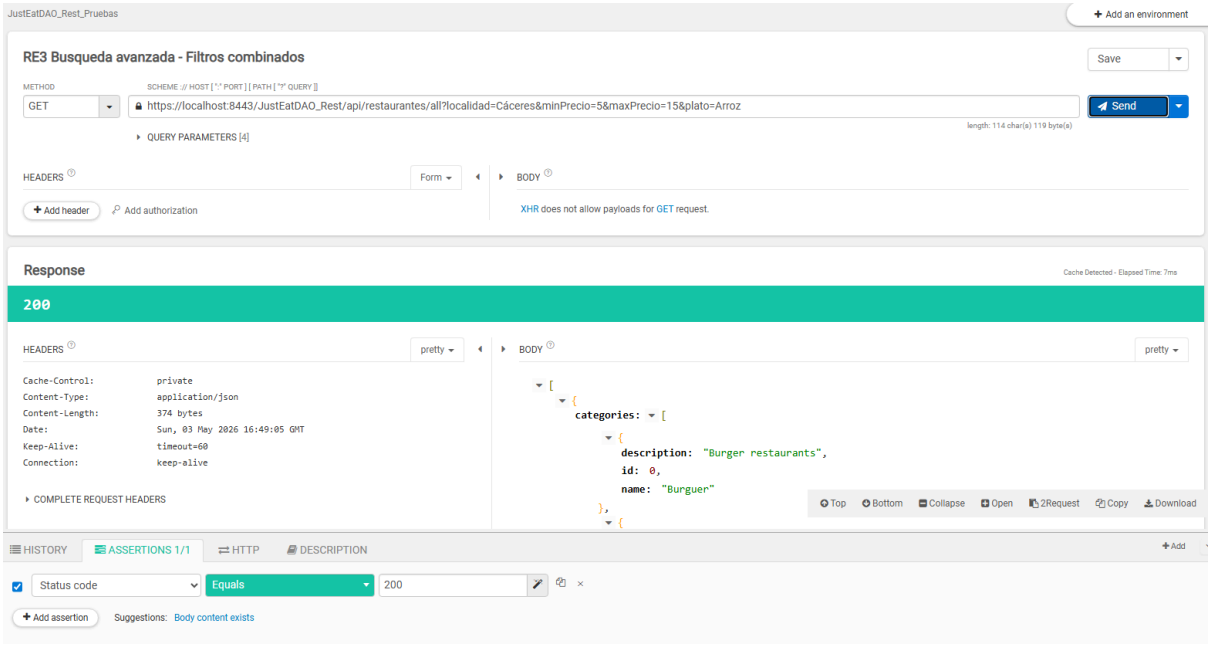
Método:
GET

Endpoint:
/api/restaurantes/all?localidad=Cáceres&minPrecio=5&maxPrecio=15&plato=Arroz

Objetivo:
Comprobar que se pueden combinar varios filtros de búsqueda avanzada.

Resultado esperado:
200 OK

Evidencia:



4.4 - Requisito Extra 4. Restaurantes relacionados

Llamadas a la API disponibles:

GET /api/restaurantes/{id}/relacionados

Prueba RE4.1 - Consultar restaurantes relacionados

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

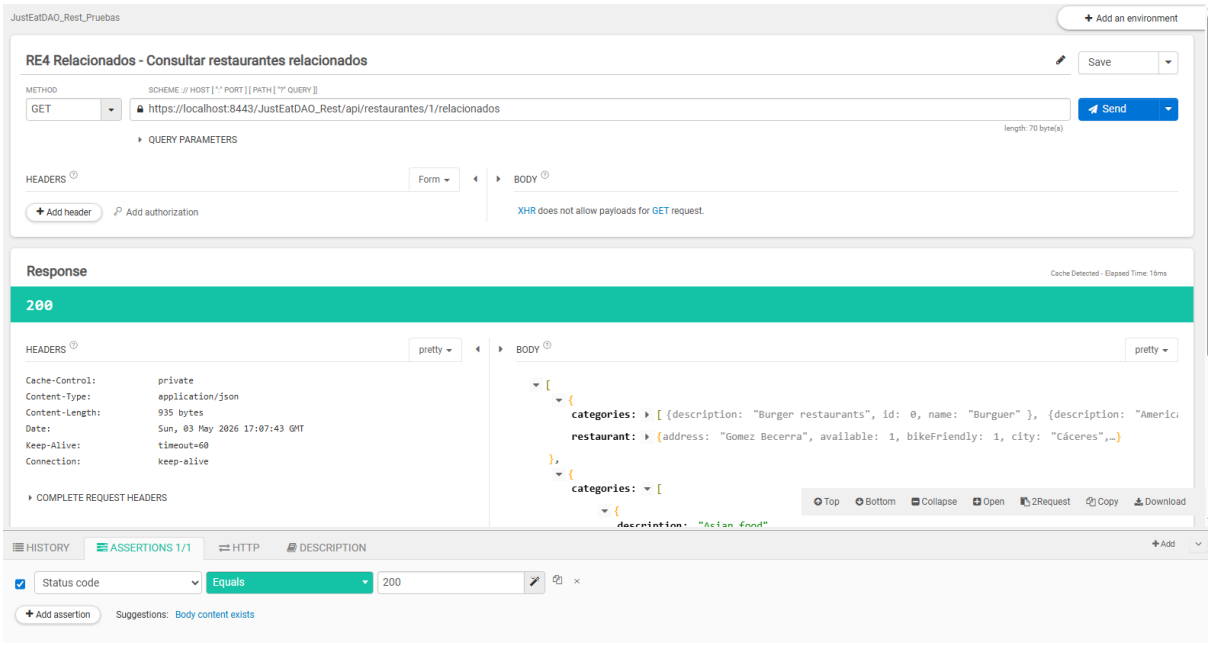
Método:
GET

Endpoint:
/api/restaurantes/1/relacionados

Objetivo:
Comprobar que la API devuelve restaurantes relacionados con el restaurante consultado.

Resultado esperado:
200 OK

Evidencia:



Prueba RE4.2 - Consultar relacionados de restaurante inexistente

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

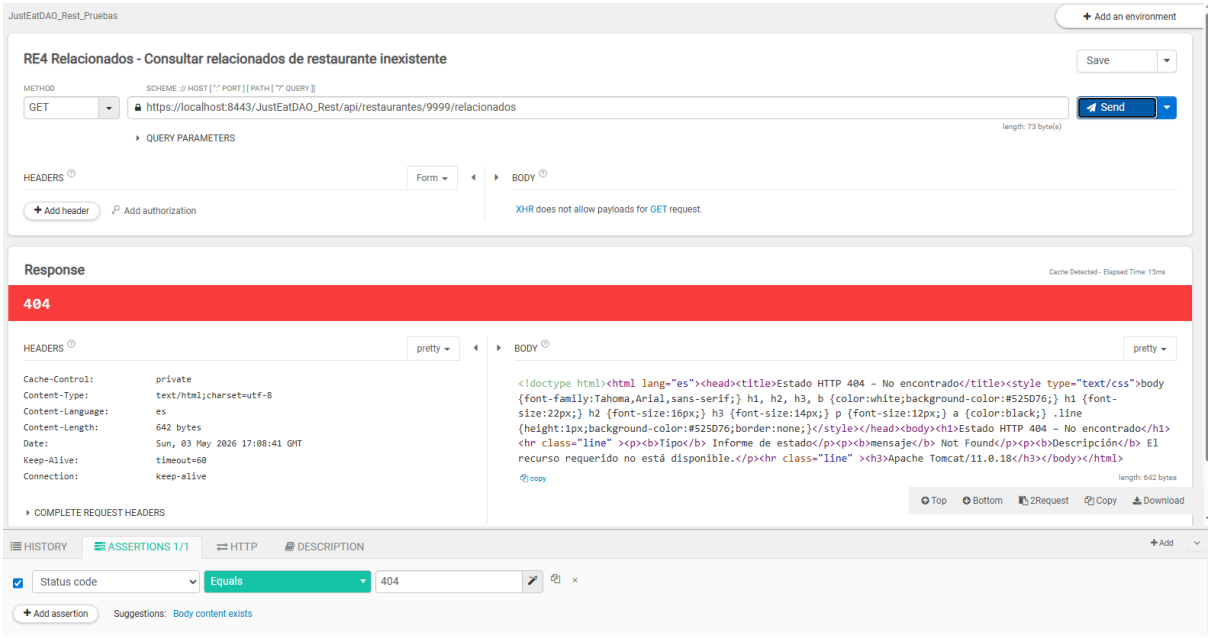
Método:
GET

Endpoint:
/api/restaurantes/9999/relacionados

Objetivo:
Comprobar que la API responde correctamente si se piden relacionados de un restaurante inexistente.

Resultado esperado:
404 Not Found

Evidencia:



4.9 - Requisito Extra 9. Panel de redes sociales

Llamadas a la API disponibles:

GET /api/restaurantes/{id}/redes
POST /api/restaurantes/{id}/redes
PUT /api/restaurantes/{id}/redes/{socialId}
DELETE /api/restaurantes/{id}/redes/{socialId}

Prueba RE9.1 - Crear restaurante con perfil social

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

Método:
POST

Endpoint:
/api/restaurantes/1/redes

Cuerpo:

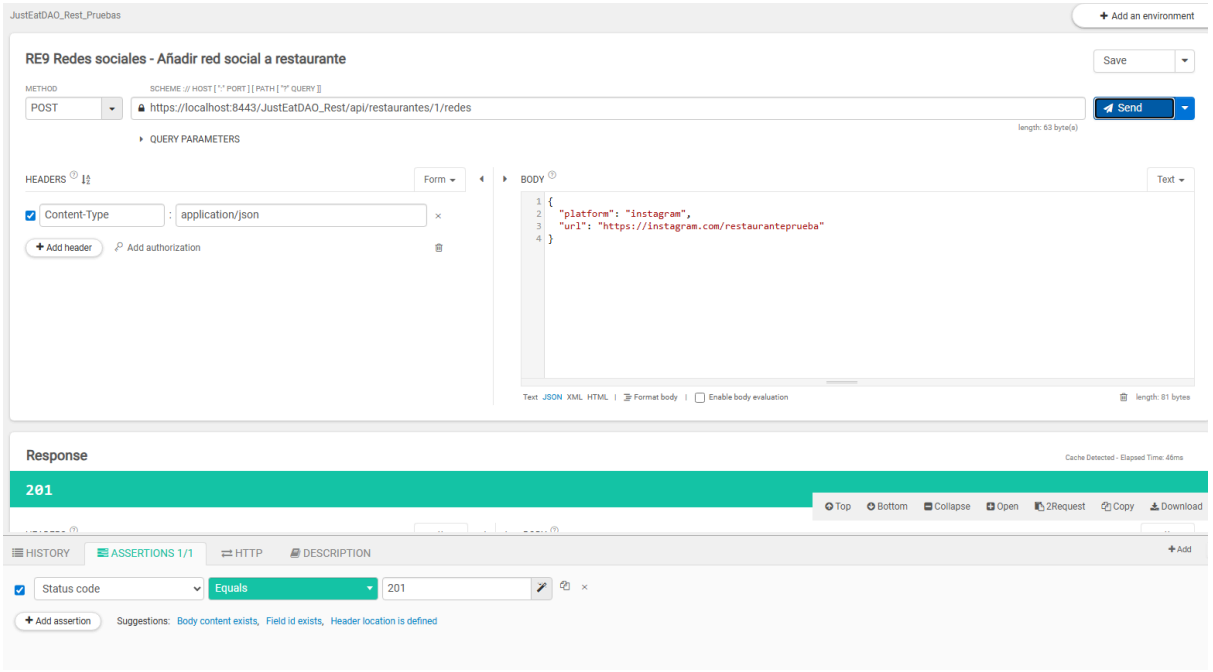
```
{
  "platform": "instagram",
```

"url": "https://instagram.com/restauranteprueba"
}

Objetivo:
Comprobar que el dueño del restaurante puede añadir una red social.

Resultado esperado:
201 Created

Evidencia:



Prueba RE9.2 - Consultar redes sociales de restaurante

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

Método:
GET

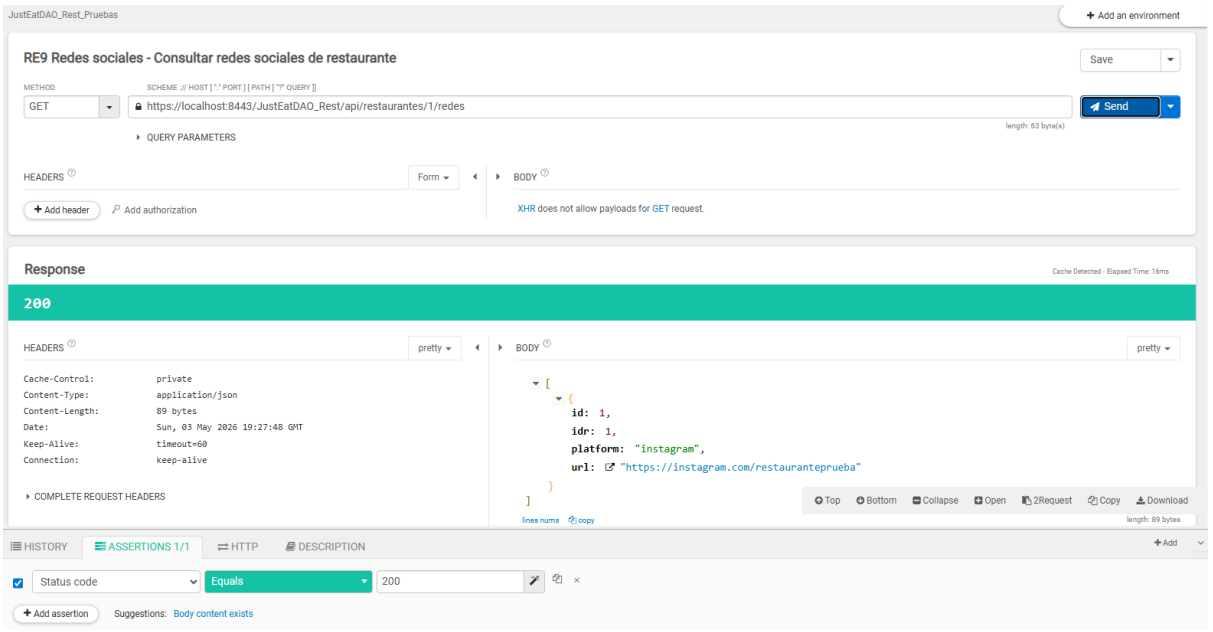
Endpoint:
/api/restaurantes/1/redes

Objetivo:

Comprobar que se pueden consultar las redes sociales asociadas a un restaurante.

Resultado esperado:
200 OK

Evidencia:



Prueba RE9.3 - Editar red social de restaurante

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

Método:
PUT

Endpoint:
/api/restaurantes/1/redes/1

Nota:
Si el ID devuelto al crear la red social no es 1, se sustituye 1 por el ID real devuelto en la respuesta del POST de creación de red social.

Body:

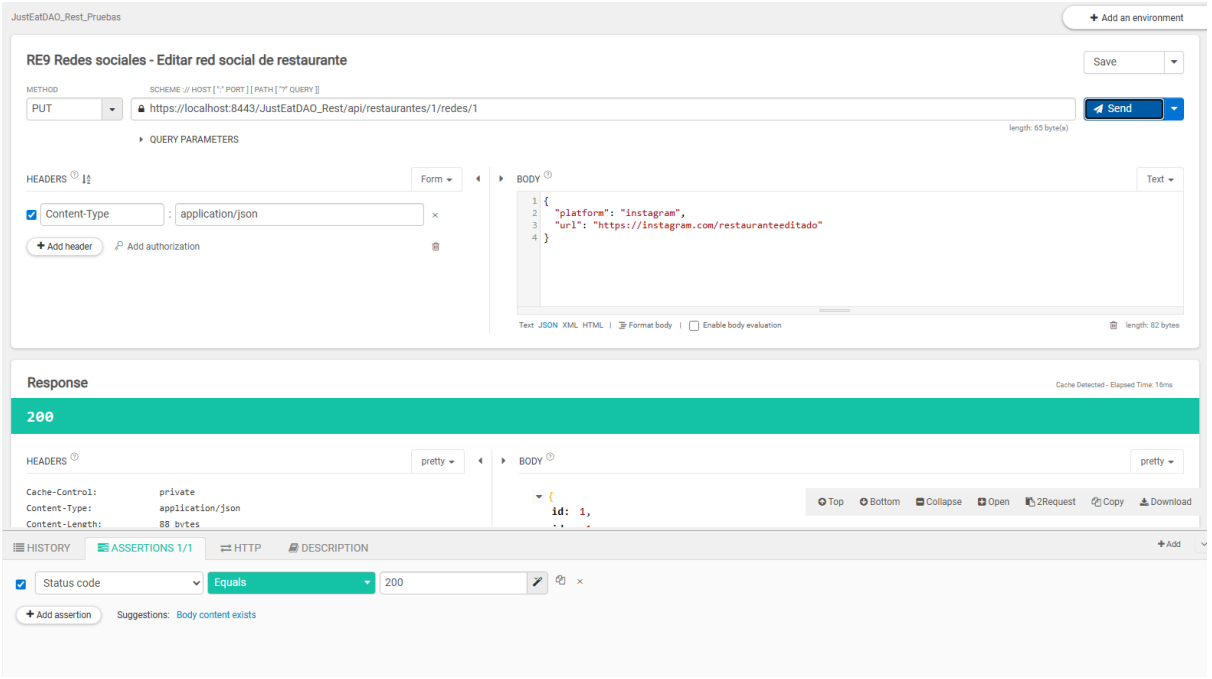
```
{
  "platform": "instagram",
```

```
  "url": "https://instagram.com/restauranteeditado"
}
```

Objetivo:
Comprobar que el dueño del restaurante puede modificar una red social existente.

Resultado esperado:
200 OK

Evidencia:



Prueba RE9.4 - Eliminar red social de restaurante

Usuario usado:
José Carlos Sancho
Email: unex@unex.es
ID: 1

Método:
DELETE

Endpoint:
/api/restaurantes/1/redes/1

Nota:

Si el ID devuelto al crear la red social no es 1, se sustituye 1 por el ID real devuelto en la respuesta del POST de creación de red social.

Objetivo:
Comprobar que el dueño del restaurante puede eliminar una red social asociada a su restaurante.

Resultado esperado:
204 No Content

Evidencia:

